



MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
INSTITUTO FEDERAL CATARINENSE - CAMPUS BLUMENAU
CURSO DE GRADUAÇÃO EM BACHAREL EM ENGENHARIA ELÉTRICA

Arthur Pedro Ferreira

Arquitetura de um Sistema Supervisório Orientada a Serviços para a Indústria 4.0

Blumenau
2025

Arthur Pedro Ferreira

Arquitetura de um Sistema Supervisório Orientada a Serviços para a Indústria 4.0

Trabalho de Conclusão de Curso de Engenharia Elétrica do Instituto Federal Catarinense, Campus Blumenau

Blumenau
2025

No meio da dificuldade, encontra-se a oportunidade.
Albert Einstein

Resumo

Página onde o autor coloca o resumo.

Palavras Chave: Palavras Chave do Trabalho

Lista de Figuras

1	Linha do Tempo das Revoluções Industriais	11
2	Pirâmide de Automação	12
3	Arquitetura típica de um sistema SCADA	13
4	Convergência entre Tecnologia da Automação (TA) e Tecnologia da Informação (TI) na automação industrial moderna	16
5	Linearização de um sensor analógico: mapeamento da unidade bruta para a unidade de engenharia, com $x_{\min} = 4000$, $x_{\max} = 20000$, $y_{\min} = 0$ bar e $y_{\max} = 10$ bar	21
6	Arquitetura em quatro camadas da IoT	23
7	Decomposição da hierarquia de automação com serviços distribuídos	25
8	Comunicação M2M em telemetria industrial: do dispositivo de campo aos serviços em nuvem	25
9	Estrutura e componentes básicos de um CPS	27
10	Interseções de IoT, CPS, IIoT e Indústria 4.0	29
11	Modelo de Arquitetura de Referência para a Indústria 4.0 (RAMI 4.0)	30
12	Modelos de serviço em computação em nuvem (pirâmide IaaS, PaaS, SaaS)	31
13	Trecho simplificado do esquema relacional do sistema supervisório	33
14	Papéis do Redis no sistema supervisório: cache de leituras quentes e barramento de eventos	34
15	Arquitetura comparada: máquinas virtuais (esquerda) e contêineres Docker (direita)	35
16	Estratégias de renderização do Next.js: CSR, SSR e SSG	36
17	Serviços AWS empregados no sistema supervisório (RDS, ECS, Lambda)	38
18	Ciclo requisição-resposta HTTP em uma API REST	40
19	Aplicação monolítica (esquerda) versus arquitetura de microserviços (direita)	41
20	Arquitetura de microserviços baseada em barramento de eventos (broker Molecular)	42
21	Composição lógica do Modbus TCP: cabeçalho MBAP seguido da PDU (código de função, dados ou exceção)	45
22	Estrutura simplificada do quadro Modbus RTU (delimitação temporal entre quadros)	45
23	Arquitetura do Sistema	47
24	Gateway ABS	52
25	Modem GPRS ABS Cel X	53

Lista de Siglas

Sigla	Significado
CLP	Controlador Lógico Programável
CNC	Controlador Numérico Computadorizado
CPS	Sistema Ciber-Físico (<i>Cyber-Physical System</i>)
ERP	<i>Enterprise Resource Planning</i> (Planejamento dos Recursos Empresariais)
I/O	Entrada/Saída (<i>Input/Output</i>)
IHM	Interface Homem-Máquina
IoT	Internet das Coisas
SCADA	Supervisão, Controle e Aquisição de Dados (<i>Supervisory Control and Data Acquisition</i>)
SOA	Arquitetura Orientada a Serviços (<i>Service Oriented Architecture</i>)
WS	Serviços Web (<i>Web Services</i>)

Lista de Simbolos

Lista de Equações

1	Equação geral da linearização do sensor	20
2	Sistema dos pontos extremos para determinação dos coeficientes	20
3	Coefficiente angular (ganho) da reta de linearização	20
4	Coefficiente linear (offset) da reta de linearização	20
5	Coefficiente angular para o sensor de pressão 0 a 10 bar	21
6	Coefficiente linear para o sensor de pressão 0 a 10 bar	21

Lista de Tabelas

1	Comparação entre IoT de consumo e IoT industrial	24
2	Cronograma de atividades do TCC.	54

Índice

1	Introdução	11
1.1	Contextualização	11
1.2	Justificativa	13
1.3	Objetivos	14
1.3.1	Objetivo Geral	14
1.3.2	Objetivos Específicos	14
1.4	Estrutura do Trabalho	14
2	Conceitos e Tecnologias Utilizadas	16
2.1	Automação e Informática Industrial	16
2.2	Sistema SCADA	17
2.2.1	Aquisição e Registro de Dados	17
2.2.2	Sincronismo de Tempo	18
2.2.3	Níveis de Operação	18
2.2.4	Comando e Controle	18
2.2.5	Escalabilidade	19
2.2.6	Redundância	19
2.3	Ajuste para Escala de Engenharia e Linearização de Sensores	19
2.3.1	Exemplo: transmissor de 4 a 20 mA com saída inteira de 4000 a 20000	20
2.4	Internet das Coisas (IoT)	22
2.5	Internet Industrial das Coisas (IIoT)	23
2.6	Comunicação Máquina a Máquina (M2M)	25
2.7	Sistemas Ciber-Físicos (CPS)	26
2.8	Indústria 4.0	28
2.9	Computação em Nuvem	30
2.10	Banco de Dados	32
2.11	Redis	33
2.12	Docker	35
2.13	Next.js	36
2.14	Fastify	37
2.15	Amazon Web Services (AWS)	38
2.16	HTTP, API REST e JSON	39
2.17	Microserviços	40
2.18	Moleculer	42
2.19	Software como Serviço (SaaS)	43
2.20	Protocolo Modbus TCP e Modbus RTU	44
2.21	Modelagem, serviço de leitura e agendamento	45
3	Funcionamento	47
3.1	Validações dos Requisitos do SCADA	47
3.1.1	Aquisição e Registro de Dados	48
3.1.2	Sincronismo de Tempo	48
3.1.3	Níveis de Operação	48
3.1.4	Comando e Controle	48
3.1.5	Escalabilidade	49
3.1.6	Redundância	49

3.2	Servidores	49
3.2.1	Frontend	50
3.2.2	Backend	50
3.2.3	Banco de Dados	50
3.2.4	Cache	50
3.2.5	APScheduler	51
3.2.6	Barramento Molecular	51
3.2.7	Workers de Aquisição	51
3.3	Aquisição de Dados	51
4	Cronograma	54

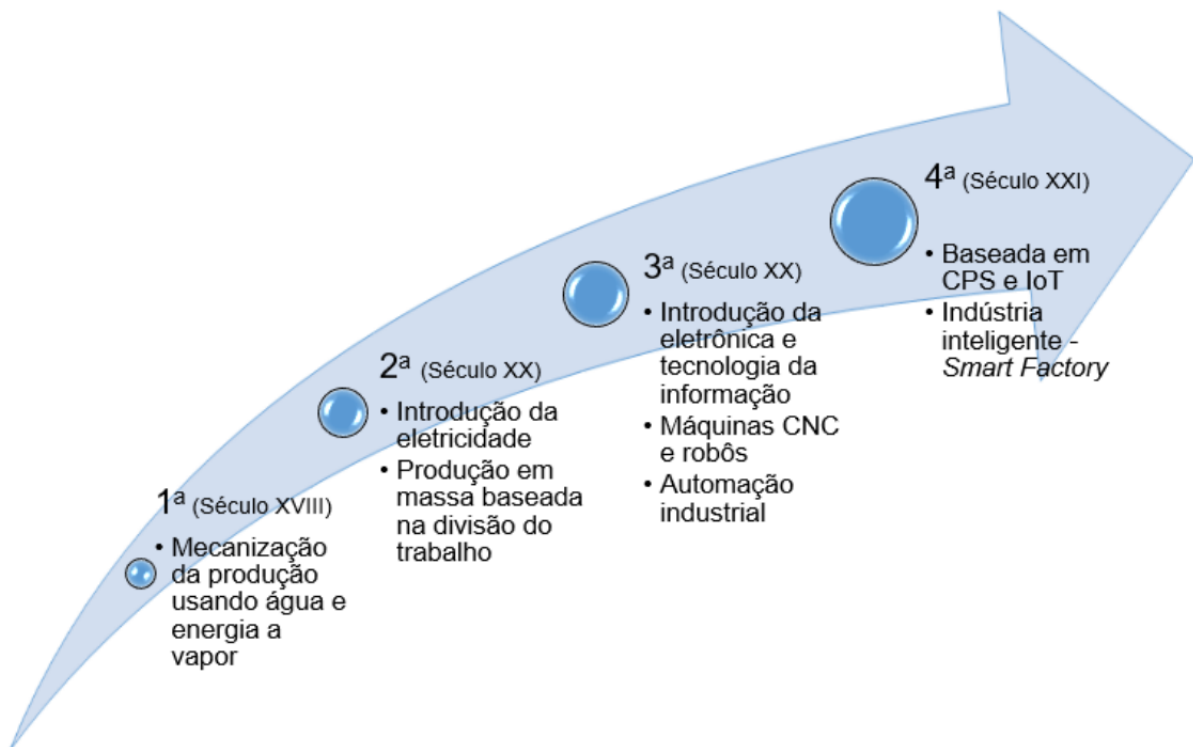
1 Introdução

1.1 Contextualização

A indústria é o setor da economia dedicado à produção de bens materiais, apoiado em trabalho humano, energia e equipamentos de alto valor agregado. Ao longo de sua evolução, sucessivos saltos tecnológicos redefiniram processos, produtos e formas de organização do trabalho, as chamadas "revoluções industriais" (Lasi *et al.*, 2014).

A Figura 1 resume essa linha do tempo. Na primeira fase, destaca-se a mecanização da produção, na segunda, a difusão da energia elétrica e a produção em massa baseada no modelo Fordista e na terceira fase, a digitalização. Foi nessa fase que a introdução da tecnologia da informação, da automação e da eletrônica impulsionou o início da automação dos processos produtivos, com o advento dos controladores lógicos programáveis (CLPs), dos Controladores Numéricos Computadorizados (CNCs) e dos robôs, tecnologias que se tornaram essenciais para as indústrias que buscam maior eficiência e qualidade com menores custos e prazos, contribuindo diretamente para o aumento da produtividade (Bigheti, 2020; Lasi *et al.*, 2014; Pisching, 2017).

Figura 1: Linha do Tempo das Revoluções Industriais



Fonte: Pisching (2017).

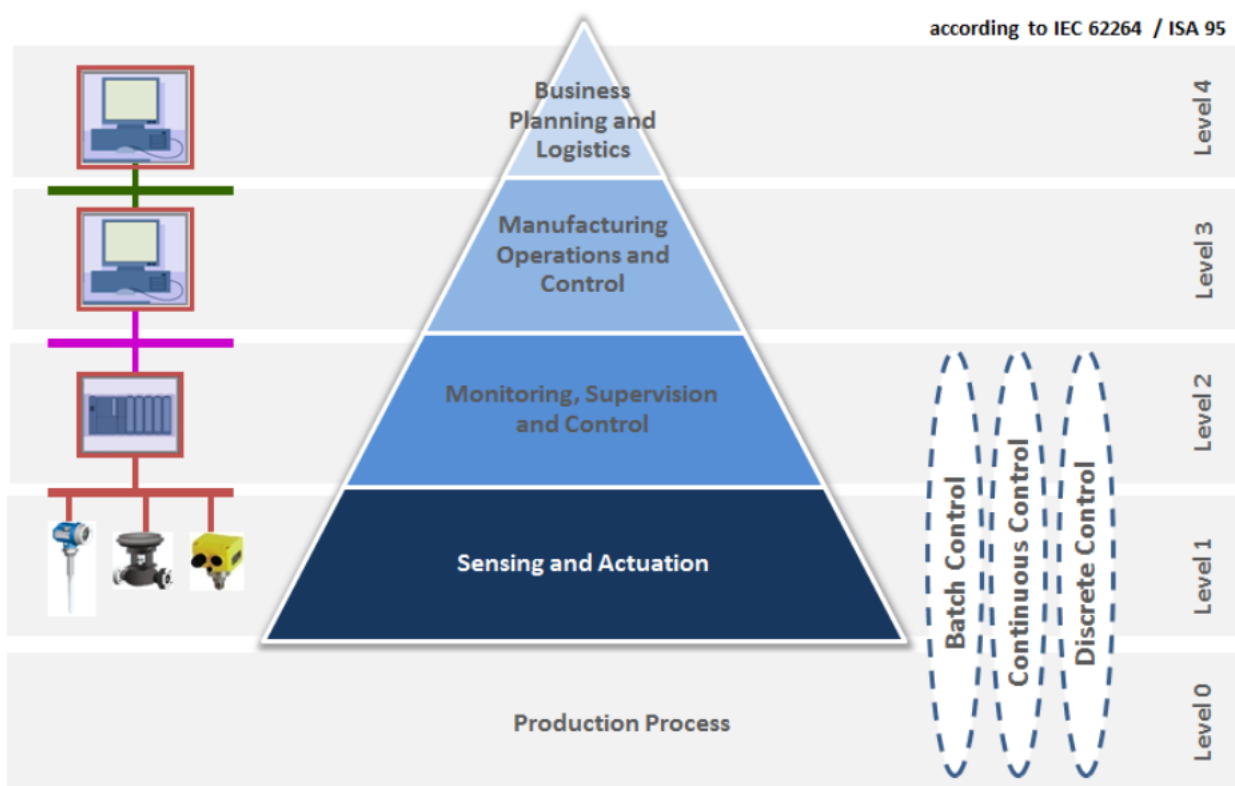
Atualmente, vive-se a quarta fase, a chamada Indústria 4.0. Baseada na indústria inteligente, na Internet das Coisas (IoT) e nos sistemas ciber-físicos (CPS - Cyber-Physical Systems), nos quais subsistemas computacionais cooperam entre si e permanecem integrados ao mundo físico e seus processos, permitindo medir, controlar e analisar dados em tempo real por meio da rede, inclusive a Internet (Bigheti, 2020; Monostori *et al.*, 2016).

A Figura 2 resume, em forma de pirâmide, como costuma ser organizada a automação na Indústria 4.0. No nível 0 está o próprio processo físico, no nível 1, sensores e atuadores,

por exemplo, um medidor de nível em malha 4-20 mA ou uma válvula de controle. No nível 2 entram os controladores lógicos programáveis (CLPs), que conversam com o nível 1 por protocolos seriais ou, de modo mais direto, pelas entradas e saídas (I/O) do equipamento. Nos níveis 3 e 4 aparecem, respectivamente, os sistemas supervisórios para acompanhar e comandar o processo em tempo real e a camada de gestão empresarial, em que se destacam os ERPs (Bigheti, 2020; Bangemann *et al.*, 2014).

Nos últimos anos, os níveis 3 e 4 foram ganhando ainda mais peso: ao digitalizar serviços antes ligados só ao ambiente físico da planta, tornou-se comum ofertá-los também sobre infraestrutura de computação em nuvem e permitir que sejam compartilhados entre diferentes sistemas e usuários. Em linhas gerais, a implementação e o gerenciamento desses serviços seguem a ideia da arquitetura orientada a serviços (Service Oriented Architecture - SOA), materializada por meio de serviços Web (Web Services - WS) (Bangemann *et al.*, 2014).

Figura 2: Pirâmide de Automação



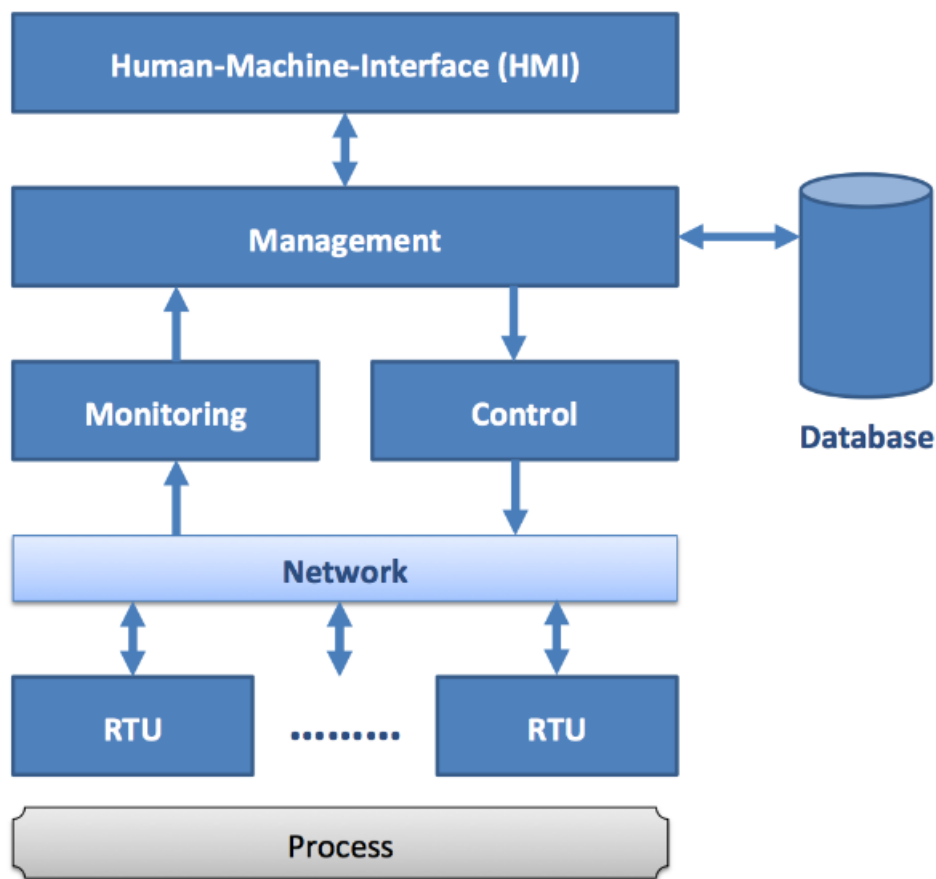
Fonte: Bangemann *et al.* (2014).

A Indústria 4.0 caracteriza-se pelo grande avanço dos protocolos de comunicação e telemetria, aliada à estar no mesmo momento de popularização do desenvolvimento de software e à interconexão de máquinas e sistemas de forma inteligente. Tais avanços permitiram o monitoramento e a supervisão centralizada de processos complexos em tempo real, elevando o papel dos sistemas supervisórios de simples interfaces de operação para centrais estratégicas de análise de dados. Nesse cenário, este trabalho propõe o desenvolvimento de um sistema supervisório web orientado a serviços.

1.2 Justificativa

Os processos industriais geram grande volume de dados, utilizados para acompanhar variáveis e interpretar a tendência do processo. Em geral, essa informação é organizada e apresentada ao operador por um software de supervisão, por meio de uma interface homem-máquina (IHM), que converte leituras de campo em telas, tendências e alarmes de forma legível e acionável. Com o aumento da capacidade de processamento dos computadores, o aprimoramento dos protocolos de comunicação e a maior integração com redes externas, inclusive a Internet, consolidou-se a supervisão remota e cresceu a exigência de visibilidade em tempo real sobre o que ocorre na planta industrial. Em setores como saneamento, transporte e energia, nos quais instalações costumam estar geograficamente espalhadas, ganham peso a telemetria e o controle supervisionado à distância. A Figura 3 ilustra, de modo esquemático, uma arquitetura típica de sistema SCADA, onde é possível ver a integração aos equipamentos de campo como CLPs, os módulos de monitoramento e controle, e a centralização da visualização dos dados para o operador (Abbas, 2011).

Figura 3: Arquitetura típica de um sistema SCADA



Fonte: Bangemann *et al.* (2014).

A computação em nuvem é um modelo em que recursos computacionais, como armazenamento, processamento, bancos de dados e aplicações, são ofertados como serviços remotos, flexíveis e escaláveis. Em geral, o usuário não precisa conhecer a localização física nem os detalhes da infraestrutura que sustenta esses serviços, o consumo costuma ser sob demanda, alinhado ao uso, o que reduz a necessidade de investimento direto em hardware e software no

ambiente local (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011). Por sua vez, a conectividade, em especial por meio de redes sem fio, é essencial para que dispositivos e sistemas ciber-físicos mantenham troca de dados com a nuvem e com demais elementos da arquitetura distribuída (Stankovic, 2014).

1.3 Objetivos

A seguir são apresentados os objetivos gerais e específicos deste Trabalho de Conclusão de Curso.

1.3.1 Objetivo Geral

Desenvolver um sistema supervisor web com arquitetura orientada a serviços, capaz de apoiar o monitoramento e a supervisão de processos industriais, com serviços bem definidos, reutilizáveis e acessíveis via interfaces web.

1.3.2 Objetivos Específicos

- Revisar os conceitos de Indústria 4.0, monitoramento industrial, arquitetura orientada a serviços (SOA) e serviços Web, de modo a embasar a solução proposta.
- Especificar os requisitos funcionais e não funcionais e a arquitetura lógica do sistema, com destaque para a separação entre camada de apresentação (web), camada de serviços e camada de obtenção ou persistência dos dados de monitoramento.
- Implementar os serviços de backend responsáveis pela leitura periódica ou sob demanda, pelo registro e pela disponibilização dos dados de processo de forma modular e interoperável.
- Implementar a interface web de monitoramento, permitindo a visualização dos principais indicadores, tendências ou telas resumo associadas ao processo acompanhado.
- Validar o módulo de monitoramento por meio de testes e de um cenário demonstrativo (ou estudo de caso), evidenciando o correto acesso às variáveis e a estabilidade da solução frente ao volume de dados esperado.

1.4 Estrutura do Trabalho

Além do capítulo 1 de introdução, o trabalho está organizado em outros 5 capítulos. No capítulo 2 são abordados os conceitos e tecnologias utilizadas dentro do trabalho, conceitos de automação industrial, tecnologia da informação industrial e desenvolvimento de software e como todos esses conceitos se relacionam para a solução proposta.

No Capítulo 3, são especificados os requisitos funcionais e não funcionais e a arquitetura do sistema, com destaque para a separação entre camada de apresentação (web), camada de serviços e camada de obtenção ou persistência dos dados de monitoramento.

No Capítulo 4, são implementados os serviços de backend responsáveis pela leitura periódica ou sob demanda, pelo registro e pela disponibilização dos dados de processo de forma modular e interoperável.

No Capítulo 5, são implementadas a interface web de monitoramento, permitindo a visualização dos principais indicadores, tendências ou telas resumo associadas ao processo acompanhado.

No Capítulo 6, são validados o módulo de monitoramento por meio de testes e de um cenário demonstrativo, evidenciando o correto acesso às variáveis e a estabilidade da solução frente ao volume de dados esperado.

2 Conceitos e Tecnologias Utilizadas

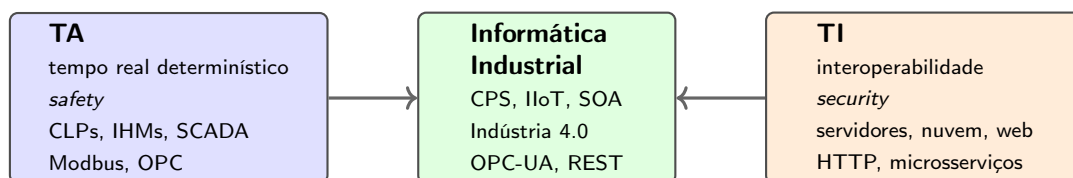
Conforme a estrutura apresentada na introdução, este capítulo aborda os conceitos e as tecnologias que sustentam o trabalho: automação industrial, tecnologia da informação aplicada ao contexto industrial e desenvolvimento de software. O objetivo é reunir definições, enquadramentos e ferramentas necessários para compreender, nas seções seguintes, a proposta de sistema supervisorio web orientado a serviços, explicitando como cada tema se articula com a solução adotada.

2.1 Automação e Informática Industrial

A **automação industrial** é o conjunto de tecnologias, técnicas e práticas que substituem a intervenção humana direta na execução de processos produtivos por sistemas eletrônicos, mecânicos e computacionais capazes de medir, controlar, comandar e supervisionar variáveis físicas com mínima interferência do operador. Em sentido amplo, abrange desde controladores eletromecânicos isolados, herdeiros da segunda revolução industrial, até as plantas inteligentes da quarta revolução discutida na Subseção 2.8, em que máquinas, produtos e sistemas se comunicam autonomamente em redes globais. A automação está presente em todos os setores produtivos relevantes: manufatura discreta (automotiva, eletroeletrônica), processos contínuos (petroquímica, papel e celulose, alimentos e bebidas), geração e distribuição de energia (elétrica, óleo e gás) e infraestrutura urbana (saneamento, transporte, edifícios).

A **informática industrial**, por sua vez, designa a aplicação das tecnologias da informação e da comunicação (TIC) ao contexto industrial. É a disciplina que articula o mundo da automação (Tecnologia da Automação, TA) com o mundo da tecnologia da informação (TI), historicamente separados por linguagens, normas, ciclos de vida e culturas profissionais distintas. Enquanto a TA preocupa-se com tempo real determinístico, robustez a ruído eletromagnético, longa vida útil dos equipamentos e segurança operacional (*safety*), a TI tradicionalmente prioriza interoperabilidade, escala, evolução rápida e segurança da informação (*security*). Bangemann *et al.* (2014) destacam que a convergência entre TA e TI é um dos principais vetores de transformação da automação contemporânea, e a Indústria 4.0 é frequentemente descrita como o desfecho dessa convergência (Pisching, 2017; Bigheti, 2020).

Figura 4: Convergência entre Tecnologia da Automação (TA) e Tecnologia da Informação (TI) na automação industrial moderna



Fonte: Elaborado pelo autor com base em Bangemann *et al.* (2014) e Bigheti (2020).

Historicamente, a evolução da automação industrial pode ser organizada em ondas tecnológicas correspondentes às revoluções industriais discutidas na Subseção 2.8. A primeira, baseada em mecanização movida a vapor e água, mecanizou tarefas físicas. A segunda, baseada na eletricidade, introduziu a produção em massa, controladores eletromecânicos (relés, temporizadores, contadores) e os primeiros barramentos de comando. A terceira, baseada em eletrônica e tecnologias da informação, popularizou os controladores lógicos programáveis (CLPs), os sistemas

de comando numérico computadorizado (CNC), os robôs industriais e, sobretudo, os sistemas de supervisão e aquisição de dados (SCADA) (Gaushell; Darlington, 1987; Zanghi, 2019). A quarta onda, em curso, baseia-se em conectividade ubíqua, sistemas ciber-físicos (Subseção 2.7) e em arquiteturas orientadas a serviços (Subseção 2.17), reposicionando o SCADA como nó de uma rede distribuída de serviços em nuvem (Bigheti, 2020; Bangemann *et al.*, 2014).

No plano normativo, a automação industrial é organizada segundo padrões que sustentam a integração entre os diferentes níveis funcionais. O mais influente é a ISA-95 (também conhecida como IEC 62264), que define a hierarquia clássica em cinco níveis: nível 0 (processo físico), nível 1 (sensoriamento e atuação), nível 2 (controle de supervisão), nível 3 (gerenciamento de operações de manufatura, com sistemas MES) e nível 4 (planejamento corporativo, com sistemas ERP). Essa pirâmide, detalhada na Subseção 2.5 ao discutir a IIoT, organiza historicamente a separação entre TA (níveis baixos, com requisitos rígidos de tempo real) e TI (níveis altos, com foco em planejamento e logística). Como observado por Bangemann *et al.* (2014) e por Monostori *et al.* (2016), essa estratificação rígida vem sendo gradualmente substituída por arquiteturas mais planas, em que serviços de diferentes níveis se compõem diretamente, viabilizadas pelos sistemas ciber-físicos e pelo paradigma de microsserviços.

O sistema supervisorio desenvolvido neste trabalho situa-se exatamente nessa fronteira entre a automação tradicional e a informática industrial moderna. No lado da TA, conecta-se a CLPs em campo por meio do protocolo Modbus (sobre RS-485 ou Ethernet), respeita as exigências de tempo real para aquisição cíclica e implementa as funções clássicas de um SCADA (sincronismo, aquisição, comando, telecontrole, alarmes, históricos). No lado da TI, expõe seus serviços em uma arquitetura web orientada a microsserviços, hospeda dados e aplicações em nuvem (Subseção 2.9), oferece interfaces para operadores e gestores via navegador, e disponibiliza APIs REST padronizadas (Subseção 2.16) para integração com outros sistemas corporativos. As subseções que se seguem detalham progressivamente cada um desses pilares, partindo da função clássica do SCADA até as tecnologias que viabilizam a operação em nuvem.

2.2 Sistema SCADA

A indústria evoluiu muito na busca por mais monitoramento e controle dos processos. Com o tempo, os custos caíram bastante para processadores, memória e discos rígidos, que levaram a redução dos custos de, por exemplo, computadores, hubs e conversores de rede, também o software foi ganhando mais peso nesse cenário. Junto a isso, cresceu a necessidade de acompanhar operações em locais remotos. As empresas de software industrial passaram a oferecer soluções com uma ampla gama de protocolos de comunicação e bem personalizadas para cada tipo de processo e indústria, a esse tipo de sistema deu-se o nome de SCADA, sigla em inglês para *Supervisory Control and Data Acquisition* (Gaushell; Darlington, 1987; Zanghi, 2019).

Para Zanghi (2019), um sistema SCADA deve conter os seguintes itens:

2.2.1 Aquisição e Registro de Dados

A aquisição das informações de campo é o primeiro grande desafio em um sistema SCADA. Ao longo das décadas, diferentes abordagens foram empregadas para trazer dados do processo até a sala de controle, evoluindo de longos cabeamentos diretos para soluções baseadas em dispositivos inteligentes próximos ao processo. Atualmente, predominam os painéis com Dispositivos Eletrônicos Inteligentes (*Intelligent Electronic Devices* - IEDs) e as Unidades Terminais Remotas (UTRs), que aquisitam grandezas analógicas e digitais (medidas de processo, estados de equipamentos, alarmes) e as disponibilizam ao supervisorio em formato binário por meio de uma rede de dados, tipicamente Ethernet (Zanghi, 2019).

Os pontos aquisitados em um sistema SCADA são denominados *tags* e armazenados no banco de dados do supervisório. Podem ser **digitais** (de entrada, DI; de saída, DO; ou de memória, DM, calculados internamente), usados para representar estados binários como liga/-desliga de motores, abertura/fechamento de válvulas e disjuntores, alarmes e eventos; ou **analógicos** (de entrada, AI; de saída, AO; ou de memória, AM), usados para grandezas multibit, como medições de pressão, vazão, nível, temperatura, tensão e corrente, comumente representadas em até 16 bits. O registro contínuo dessas informações pelo supervisório permite análises posteriores na forma de gráficos históricos de tendência e de sequenciais de eventos digitais (Zanghi, 2019).

2.2.2 Sincronismo de Tempo

Outro requisito fundamental dos sistemas SCADA é o sincronismo de tempo. Todo dado coletado deve possuir um registro de tempo de origem, a chamada estampa de tempo (*time stamp*): cada variação de um sinal digital, seja para “0” ou “1”, precisa ser registrada com o instante exato em que ocorreu. Essa exigência decorre da necessidade de análises *a posteriori* de ocorrências e faltas, que dependem da cronologia precisa dos acontecimentos para permitir a identificação das causas, sobretudo em processos com dinâmica rápida (Zanghi, 2019).

Para garantir essa sincronização, é comum o uso de equipamentos receptores de sinal de GPS conectados aos IEDs e ao próprio supervisório, ajustando seus relógios internos com precisão de pelo menos 1 ms. Assim, alarmes e eventos digitais captados em diferentes pontos do sistema recebem estampas de tempo coerentes, sendo essa informação transportada ao supervisório pelo protocolo de comunicação empregado entre os dispositivos (Zanghi, 2019).

2.2.3 Níveis de Operação

Os sistemas SCADA são tipicamente organizados em diferentes níveis de operação, cada um atendendo a necessidades distintas e operando sobre um subconjunto dos pontos disponíveis nos níveis inferiores. No **Nível 0** estão os equipamentos de campo e seus sensores, fonte primária dos dados. O **Nível 1** corresponde à supervisão local, próxima aos painéis dos IEDs, normalmente implementada em uma Interface Homem-Máquina (IHM) simples, voltada à operação no próprio local da planta e sem necessidade de arquitetura redundante (Zanghi, 2019).

O **Nível 2** representa a supervisão da instalação como um todo, agregando os pontos relevantes dos IEDs em estações de trabalho robustas, geralmente redundantes, com telas mais elaboradas que permitem supervisão e controle completos da unidade. Por fim, o **Nível 3** é o centro de operação remoto, que consolida dados provenientes de múltiplas instalações distribuídas geograficamente; nele apenas os pontos estritamente necessários à operação sistêmica são disponibilizados, exigindo otimização do volume de dados e alta robustez para funcionamento contínuo. À medida que se sobe na hierarquia, a quantidade de pontos é filtrada e o foco passa do controle local detalhado para a gestão integrada da operação (Zanghi, 2019).

2.2.4 Comando e Controle

As ações de operação executadas remotamente pelo supervisório são denominadas telecomando e telecontrole. O software SCADA deve prever três tipos básicos: o **telecomando instantâneo**, voltado à mudança rápida de estados de equipamentos com acionamento remoto (e.g., partida de uma bomba, abertura de uma válvula, comando a um disjuntor); o **telecontrole**, destinado a regular no tempo a resposta de um processo a partir de um valor desejado (*set point*); e o **telecomando ou telecontrole programado**, que permite o agendamento de

ações conforme condições ou cenários previamente definidos, registrando o instante de execução (Zanghi, 2019).

O processamento de um telecomando segue quatro fases distintas: **seleção**, em que o operador escolhe na interface o equipamento a ser comandado; **verificação** dos elementos envolvidos, como intertravamentos e condições de segurança; **confirmação** pela ação explícita do operador; e **execução**, com o envio efetivo da ordem ao dispositivo responsável pelo elemento final de controle. Após a execução, é boa prática que o supervisor realize a verificação de confirmação, garantindo que a solicitação foi aceita e processada pelo dispositivo de campo; em caso de falha, um alarme correspondente deve ser gerado para sinalizar a inconsistência ao operador (Zanghi, 2019).

2.2.5 Escalabilidade

A expansão das instalações é uma realidade frequente em ambientes industriais, com inclusão de novos equipamentos, pontos de medição e circuitos. Para acompanhar essas mudanças, o software SCADA deve ser escalável, permitindo o acréscimo de pontos em sua base de dados e a criação de novas telas de operação sem comprometer o que já está em produção. A maioria dos supervisórios comerciais oferece essa capacidade, contudo as licenças associadas precisam contemplar a expansão prevista. Licenças com número ilimitado de pontos e telas existem, mas têm custo elevado, o que torna a análise correta da escalabilidade fundamental para o adequado dimensionamento e orçamento da solução (Zanghi, 2019).

Quando se considera o sistema SCADA como um todo, a análise da escalabilidade deve abranger não apenas o supervisor, mas também os elementos de aquisição, controle e rede. Os *hardwares* envolvidos devem se manter íntegros em suas funções e permitir o acréscimo de novos dispositivos ao barramento de comunicação, o que torna essencial que essa integração seja prevista já na fase inicial do projeto de automação (Zanghi, 2019).

2.2.6 Redundância

A redundância consiste no acréscimo de um elemento idêntico ao original, de modo que, em caso de falha do primeiro, o segundo assuma as funções da maneira mais transparente possível para a operação. Essa prática se aplica a diversos componentes do sistema SCADA, como softwares supervisórios, dispositivos de aquisição e controle, e elementos de rede (por exemplo, *switches*); apesar do custo adicional, permite alcançar graus elevados de confiabilidade, compatíveis com a criticidade exigida em aplicações industriais sensíveis (Zanghi, 2019).

Em uma arquitetura redundante típica, múltiplas estações de operação comunicam-se simultaneamente com os dispositivos de campo, garantindo que a perda de uma delas não impacte a atualização das informações; *switches* interligados em topologia em anel, com portas redundantes, asseguram resiliência a falhas de cabeamento e de portas individuais; e as unidades de aquisição e controle podem operar em pares espelhados, mantendo comunicação duplicada com os *switches* e com os dispositivos finais. Esse conjunto de medidas garante a continuidade da supervisão e do controle mesmo diante de falhas pontuais em equipamentos ou enlaces de rede (Zanghi, 2019).

2.3 Ajuste para Escala de Engenharia e Linearização de Sensores

Em sistemas de aquisição de dados, raramente o valor lido diretamente do registrador do CLP ou do conversor analógico-digital corresponde, em forma e magnitude, à grandeza física que se deseja monitorar. Sensores industriais entregam sinais elétricos padronizados (por exemplo,

correntes de 4 a 20 mA ou tensões de 0 a 10 V) que, depois de digitalizados pelo CLP, ficam disponíveis como inteiros em uma faixa numérica fixa, a chamada **unidade bruta** (*raw*). Cabe ao sistema supervisorio, ou à camada de aquisição, converter esse valor bruto para a **unidade de engenharia** apropriada à operação (metro, m³/h, bar, °C, kW etc.), por meio de um procedimento conhecido como *ajuste para a escala de engenharia* ou, de forma equivalente, *linearização do sensor*.

Considere um sensor cuja saída elétrica é proporcional à grandeza medida, caso típico de transmissores industriais de pressão, vazão, nível, temperatura e corrente. Nessa situação, a relação entre o valor bruto x obtido pelo CLP e o valor de engenharia y entregue à supervisão é uma **função afim**, da forma:

$$y = a \cdot x + b \quad [\text{u.e.}] \quad (1)$$

onde a é o coeficiente angular, também chamado de *ganho* ou *fator de escala*, e b é o coeficiente linear, também chamado de *offset* ou *deslocamento*.

Sejam $x_{\min}$ e $x_{\max}$ os limites inferior e superior da unidade bruta (a faixa numérica que o CLP entrega), e $y_{\min}$ e $y_{\max}$ os limites inferior e superior da unidade de engenharia (a faixa física que o sensor cobre). A reta da Equação 1 deve passar pelos dois pontos extremos $(x_{\min}, y_{\min})$ e $(x_{\max}, y_{\max})$. Substituindo cada par na equação obtém-se o sistema:

$$\begin{cases} y_{\min} = a \cdot x_{\min} + b \\ y_{\max} = a \cdot x_{\max} + b \end{cases} \quad [\text{u.e.}] \quad (2)$$

Subtraindo a primeira equação da segunda, elimina-se b , obtendo $y_{\max} - y_{\min} = a \cdot (x_{\max} - x_{\min})$, e isola-se a :

$$a = \frac{y_{\max} - y_{\min}}{x_{\max} - x_{\min}} \quad [\text{u.e.}] \quad (3)$$

Substituindo o valor de a em qualquer uma das duas equações originais, determina-se b :

$$b = y_{\min} - a \cdot x_{\min} \quad [\text{u.e.}] \quad (4)$$

As Equações 3 e 4 fornecem uma forma fechada para o ajuste de escala: conhecidos os quatro limites $(x_{\min}, x_{\max}, y_{\min}$ e $y_{\max})$, basta calcular uma única vez os coeficientes a e b no momento da configuração do ponto, e cada leitura bruta posterior é convertida em valor de engenharia por uma multiplicação seguida de uma soma, operações leves o suficiente para serem aplicadas a milhares de *tags* sem comprometer o desempenho da aquisição.

2.3.1 Exemplo: transmissor de 4 a 20 mA com saída inteira de 4000 a 20000

O caso mais frequente em automação industrial é o do transmissor de corrente de 4 a 20 mA. Esse padrão, conhecido como *loop* de corrente, é robusto a ruído elétrico e permite detecção de rompimento de cabo, pois uma corrente abaixo de 4 mA indica falha de medição ou rompimento do enlace. Quando o sinal é amostrado por um cartão de entrada analógica do CLP com resolução adequada e o programa do CLP grava o valor como inteiro em ponto fixo com duas casas decimais de precisão sobre os miliampères, a corrente é representada na

unidade bruta como um número inteiro de 4000 a 20000. Isto é, 4,00 mA aparece como o inteiro $x_{\min} = 4000$ e 20,00 mA aparece como o inteiro $x_{\max} = 20000$.

Suponha um sensor de pressão calibrado para uma faixa de engenharia de $y_{\min} = 0$ bar a $y_{\max} = 10$ bar. Aplicando a Equação 3:

$$a = \frac{10 - 0}{20000 - 4000} = \frac{10}{16000} = 6,25 \times 10^{-4} \quad [\text{bar}/\text{bruta}] \quad (5)$$

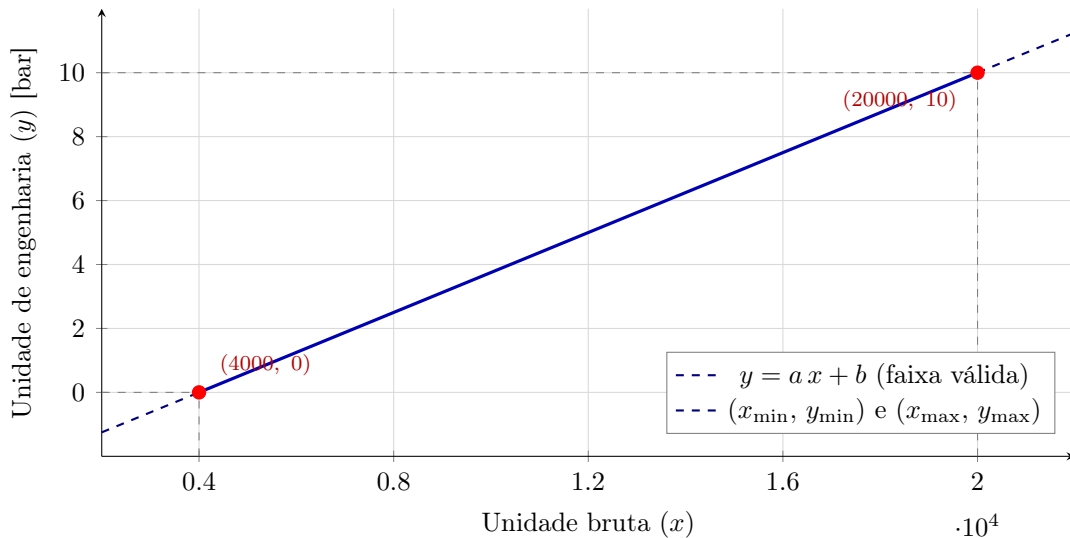
e a Equação 4:

$$b = 0 - 6,25 \times 10^{-4} \cdot 4000 = -2,5 \quad [\text{bar}] \quad (6)$$

Logo, a equação de linearização para esse ponto específico fica $y = 6,25 \times 10^{-4} x - 2,5$. Uma leitura bruta de 12000, por exemplo, corresponde a $y = 6,25 \times 10^{-4} \cdot 12000 - 2,5 = 5,0$ bar, valor coerente com o ponto médio da faixa.

A Figura 5 ilustra graficamente o mapeamento: o eixo das abscissas representa a unidade bruta no intervalo de $x_{\min} = 4000$ a $x_{\max} = 20000$, e o eixo das ordenadas representa a unidade de engenharia, com $y_{\min}$ e $y_{\max}$ marcados nos extremos. A reta resultante atravessa exatamente os dois pontos $(x_{\min}, y_{\min})$ e $(x_{\max}, y_{\max})$, garantindo que qualquer leitura bruta intermediária seja mapeada de forma única e proporcional para a unidade física desejada. Os trechos tracejados nas extremidades indicam o prolongamento matemático da reta fora da faixa válida de operação do sensor, onde a relação ainda é matematicamente definida, mas não corresponde a valores fisicamente garantidos pela calibração.

Figura 5: Linearização de um sensor analógico: mapeamento da unidade bruta para a unidade de engenharia, com $x_{\min} = 4000$, $x_{\max} = 20000$, $y_{\min} = 0$ bar e $y_{\max} = 10$ bar



Fonte: Autor.

A generalização é direta: o procedimento descrito não se restringe ao padrão 4 a 20 mA nem à faixa bruta de 4000 a 20000. A mesma metodologia se aplica a qualquer sensor cuja saída seja linearmente proporcional à grandeza medida, alterando-se apenas os quatro limites. Como exemplos comuns na prática industrial:

- Sensores de tensão de 0 a 10 V representados como inteiros de 0 a 10000;
- Termopares com saída digital em décimos de grau, com faixa bruta de -2000 a $+12000$ representando -200°C a $+1200^{\circ}\text{C}$;
- Medidores de vazão com saída em pulsos acumulados, em que o ajuste de escala converte a contagem para metros cúbicos por hora;
- Sensores ultrassônicos de nível com faixa bruta de 0 a 16383 (resolução de 14 bits) mapeada para 0 a 5 m de coluna d'água.

Vale ressaltar duas situações em que essa abordagem precisa ser estendida. A primeira é o caso em que a relação entre a grandeza física e o sinal elétrico não é linear: termopares, por exemplo, possuem curva tensão-temperatura tabelada por norma e, na prática, são linearizados por aproximações polinomiais por trechos ou por tabelas *lookup* já internas ao próprio CLP, de forma que o supervisor recebe um valor já compensado e aplica apenas o ajuste linear de escala descrito nesta subseção. A segunda é o caso em que o sensor exige, além do ajuste de escala, correções de *zero* e *span* para compensar derivas de calibração ao longo do tempo; nessa situação, a e b não são fixos, mas atualizáveis pela rotina de manutenção, sem que isso altere a estrutura matemática descrita pela Equação 1.

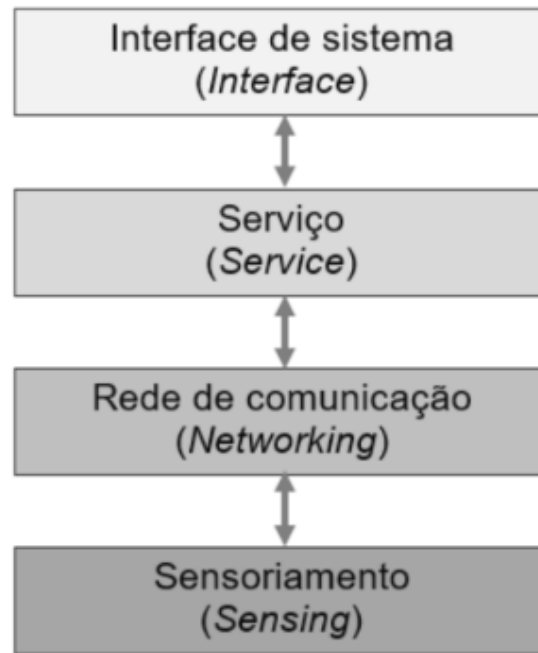
2.4 Internet das Coisas (IoT)

O termo *Internet of Things* (IoT), ou Internet das Coisas, foi cunhado em 1999 por Kevin Ashton no contexto de gerenciamento de cadeias de suprimento e, desde então, expandiu-se para áreas tão diversas quanto saúde, transporte, habitação, agricultura e indústria (Pisching, 2017; Bigheti, 2020). O paradigma descreve uma infraestrutura de rede global e dinâmica, baseada em protocolos de comunicação interoperáveis, na qual “coisas” físicas e virtuais possuem identificadores únicos, atributos próprios e uma representação digital, capazes de se integrar à internet por meio de interfaces inteligentes (Pisching, 2017). Em termos práticos, trata-se de um mundo de objetos embarcados com sensores e atuadores, conectados em rede, frequentemente sem fio, que trocam informações de forma autônoma e reagem a estímulos do ambiente (Bigheti, 2020).

Stankovic (2014) argumenta que a IoT representa um ponto de convergência entre comunidades de pesquisa historicamente distintas, computação móvel, computação pervasiva, redes de sensores sem fio e sistemas ciber-físicos, cujas fronteiras se tornaram cada vez mais tênues à medida que sensoramento, atuação, comunicação e controle se tornam ubíquos. A visão associada à IoT prevê uma transformação qualitativa quando a densidade de sensoramento triplicar ou quadruplicar em relação à atualmente observada: cidades, edifícios e equipamentos passarão a ser permeados por sensores e atuadores que, em conjunto, configurarão uma “utilidade” (*utility*) semelhante à eletricidade ou à água, sobre a qual aplicações personalizadas, urbanas e globais poderão ser executadas (Stankovic, 2014).

A evolução da IoT pode ser organizada, segundo Pisching (2017), em três fases: a primeira, ligada à identificação automática por código de barras e RFID, voltada à automação de inventário e ao rastreamento de objetos; a segunda, atualmente em curso, marcada pela interconexão massiva de sensores, dispositivos, dados e aplicações; e a terceira, denominada “IoT cognitiva”, em que a interoperabilidade semântica permitirá reutilizar informações de objetos e ampliar a capacidade de inferência das aplicações. A Figura 6 sintetiza a arquitetura em quatro camadas comumente adotada para descrever sistemas IoT, sensoramento, rede, serviço e interface, na qual cada camada agrega abstração crescente sobre os dados originados no nível físico.

Figura 6: Arquitetura em quatro camadas da IoT



Fonte: Pisching (2017, p. 72).

A escala antevista para a IoT, da ordem de trilhões de dispositivos conectados à internet, impõe desafios técnicos significativos: como nomear, autenticar, manter e proteger esse universo de objetos; como suportar real-time e confiabilidade em redes heterogêneas; como armazenar e extrair conhecimento das massas de dados gerados; e como conciliar abertura (*openness*) das aplicações com segurança e privacidade (Stankovic, 2014). Em particular, a interação cyber-físico introduzida pela IoT exige novas técnicas de análise: como os dispositivos atuam sobre o mundo físico, falhas podem ter consequências de segurança operacional, o que demanda inferência confiável a partir de dados ruidosos e modelos de robustez para sistemas de longa duração e operação não assistida (Stankovic, 2014).

2.5 Internet Industrial das Coisas (IIoT)

Quando o paradigma da IoT é trazido para o domínio industrial, sensoriamento de processos, controle de máquinas e otimização de cadeias produtivas, recebe a denominação *Industrial Internet of Things* (IIoT). A IIoT é, em essência, um subconjunto da IoT especializado para aplicações industriais, no qual a interconexão de sensores, atuadores, equipamentos e sistemas de produção visa simplificar arquiteturas, ampliar a eficiência e tornar as plantas mais responsivas às demandas de operação (Bigheti, 2020). Para Pisching (2017), a rápida proliferação da IoT no setor industrial vem transformando processos e infraestrutura, levando fabricantes de equipamentos de automação a investir em tecnologias embarcadas que suportem essa convergência entre tecnologia da informação (TI) e tecnologia da automação (TA).

Embora compartilhem o mesmo núcleo conceitual, a interconexão de objetos via redes baseadas em IP, IoT de consumo e IIoT diferem significativamente em requisitos técnicos, modelo de serviço e criticidade. A IoT de consumo está orientada a pessoas e cenários ad-hoc, com volume médio de dados e requisitos brandos de tempo de resposta e segurança; a IIoT, por sua vez, lida com nós fixos, redes estruturadas, volumes elevados de dados e requisitos rigorosos

de confiabilidade, tempo real e segurança operacional, dado que falhas podem comprometer processos produtivos críticos (Bigheti, 2020). A Tabela 1 sintetiza esse contraste.

Tabela 1: Comparação entre IoT de consumo e IoT industrial

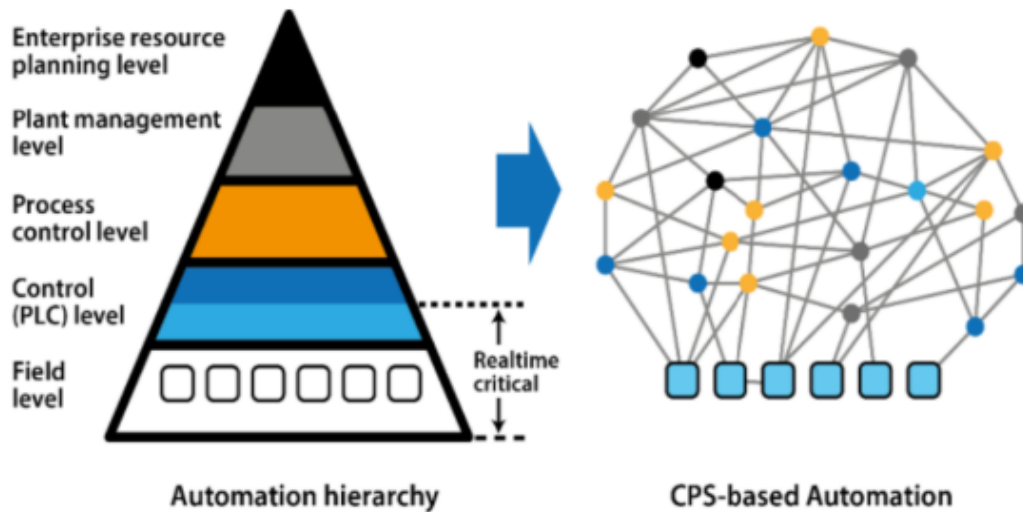
Critério	IoT de consumo	IoT industrial (IIoT)
Impacto	Revolução	Evolução
Modelo de serviço	Voltada ao ser humano	Orientada às máquinas
Estado atual	Novos dispositivos e padrões emergindo	Dispositivos e padrões já consolidados
Conectividade	Não estruturada, ad-hoc ou móvel	Estruturada, com nós fixos e gerenciamento centralizado de rede
Criticidade	Pouco rigorosa	Rigorosa em tempo, confiabilidade, segurança e privacidade
Volume de dados	Médio a alto	Alto a muito alto

Fonte: Adaptado de Bigheti (2020), com base em Sisinni et al. (2018).

No contexto da indústria, a IIoT viabiliza três paradigmas operacionais frequentemente associados à transformação digital da manufatura: o **produto inteligente** (*smart product*), que carrega informações sobre si mesmo e sobre seu processo produtivo, permitindo rastreabilidade ao longo de todo o ciclo de vida; a **máquina inteligente** (*smart machine*), capaz de se auto-configurar, coordenar-se com outras máquinas e oferecer funcionalidades por meio de serviços de rede; e o **operador aumentado** (*augmented operator*), apoiado por dispositivos vestíveis, *tablets* e óculos inteligentes que ampliam a percepção e a capacidade de intervenção sobre o processo (Bigheti, 2020).

A IIoT impacta diretamente a arquitetura tradicional da automação industrial, organizada historicamente em camadas hierárquicas segundo a norma ISA-95/IEC 62264 (processo, controle, supervisão, MES e ERP). Bangemann *et al.* (2014) observam que, à medida que dispositivos de campo expõem suas funcionalidades como serviços de rede (por exemplo, via OPC-UA ou DPWS), a pirâmide ISA-95 tende a se “achatar” em uma arquitetura logicamente plana baseada em serviços, na qual um serviço exposto por uma válvula no nível de campo pode se compor diretamente com um serviço executado em um sistema MES (Figura 7). Essa transição é um dos vetores tecnológicos centrais da indústria 4.0, discutida na Seção 2.8.

Figura 7: Decomposição da hierarquia de automação com serviços distribuídos



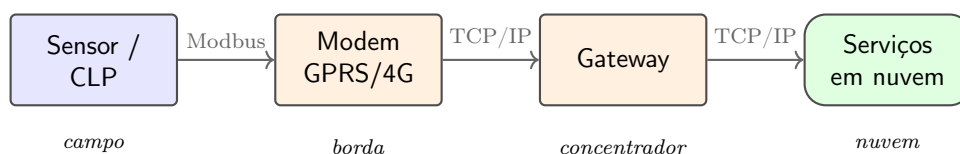
Fonte: Monostori *et al.* (2016).

2.6 Comunicação Máquina a Máquina (M2M)

A comunicação **Máquina a Máquina** (*Machine-to-Machine*, M2M) designa a troca direta de informações entre dispositivos físicos sem intervenção humana imediata. Originada no setor de telemetria industrial e nos primeiros sistemas SCADA, em que sensores remotos transmitiam leituras para uma estação central por meio de enlaces dedicados, a M2M precede e fundamenta historicamente a Internet das Coisas (Subseção 2.4): enquanto a IoT é frequentemente associada a um cenário amplo e heterogêneo, com bilhões de objetos cotidianos conectados, a M2M, em sua acepção mais antiga, refere-se especificamente à comunicação ponto a ponto entre dispositivos industriais por meios determinísticos como redes celulares dedicadas, rádio, satélite ou cabeamento ponto a ponto.

No contexto da Indústria 4.0, a M2M é a base de comunicação que viabiliza a noção de *Máquina Inteligente* (*Smart Machine*) descrita por Bigheti (2020) e Pisching (2017): dispositivos de campo que não apenas executam funções predefinidas, mas trocam informações com outros dispositivos, coordenam ações, ajustam parâmetros operacionais e reportam estados a sistemas de supervisão sem intervenção manual. Em uma planta industrial moderna, isso se traduz em CLPs que conversam diretamente com outros CLPs para sincronizar operações entre linhas de produção, inversores de frequência que negociam com sensores de processo para ajustar velocidade de bombas em função de medições de vazão, e medidores que reportam valores periódicos a múltiplas aplicações simultaneamente.

Figura 8: Comunicação M2M em telemetria industrial: do dispositivo de campo aos serviços em nuvem



Fonte: Elaborado pelo autor com base em Bigheti (2020) e Pisching (2017).

Tecnologicamente, a M2M industrial é sustentada por uma família de protocolos de comunicação, cada qual adequado a uma fatia de aplicações. O **Modbus** (Subseção 2.20) é o protocolo industrial mais difundido em telemetria, particularmente em sua versão sobre TCP/IP, que permite encapsular requisições entre CLPs distantes. O **MQTT** (*Message Queuing Telemetry Transport*) é um protocolo publicador-assinante leve, projetado para ambientes com banda limitada e dispositivos restritos, amplamente adotado em IoT e em telemetria industrial moderna. O **OPC-UA** (*Open Platform Communications Unified Architecture*), discutido por Bangemann *et al.* (2014), oferece um modelo unificado de informação para comunicação segura entre dispositivos heterogêneos, com forte adoção nas iniciativas mais recentes de Indústria 4.0. Outros protocolos, como CoAP, HTTP/REST (Subseção 2.16) e gRPC, encontram nicho em integrações específicas conforme requisitos de banda, latência e segurança.

No sistema supervisorio proposto neste trabalho, a M2M materializa-se em uma cadeia bem definida (Figura 8). No campo, os CLPs operam protocolo Modbus, expondo seus registradores e *coils* a clientes remotos. Localmente, um modem GPRS ou 4G concentra a comunicação com o equipamento e estabelece a sessão WAN com o *gateway* no provedor. O *gateway* agrega múltiplas sessões e roteia requisições conforme cadastro, em um esquema discutido em detalhe na Subseção 2.20. Por fim, a aplicação em nuvem dispara as requisições de leitura segundo o agendador de aquisição, recebe as respostas e converte os valores brutos para unidade de engenharia (Subseção 2.3) antes de publicá-los aos serviços consumidores. Essa arquitetura permite que centenas de pontos remotos sejam supervisionados a partir de um único conjunto de serviços de nuvem, sem necessidade de hardware proprietário em cada cliente.

Vale notar que a M2M também sustenta tendências de fronteira na automação contemporânea. A **computação na borda** (*edge computing*) descentraliza parte do processamento para os próprios dispositivos de campo ou para concentradores próximos, reduzindo latência e tráfego para a nuvem. A **computação na névoa** (*fog computing*) introduz camadas intermediárias entre borda e nuvem para tarefas com requisitos mistos de latência e capacidade. Modelos de **telemetria orientada a eventos** substituem o *polling* cíclico por publicação assíncrona quando o valor amostrado muda significativamente, reduzindo consumo de banda e energia em dispositivos alimentados por bateria. Embora essas tendências estejam fora do escopo principal do presente trabalho, são extensões naturais da arquitetura M2M aqui adotada e podem ser incorporadas em iterações futuras do sistema.

2.7 Sistemas Ciber-Físicos (CPS)

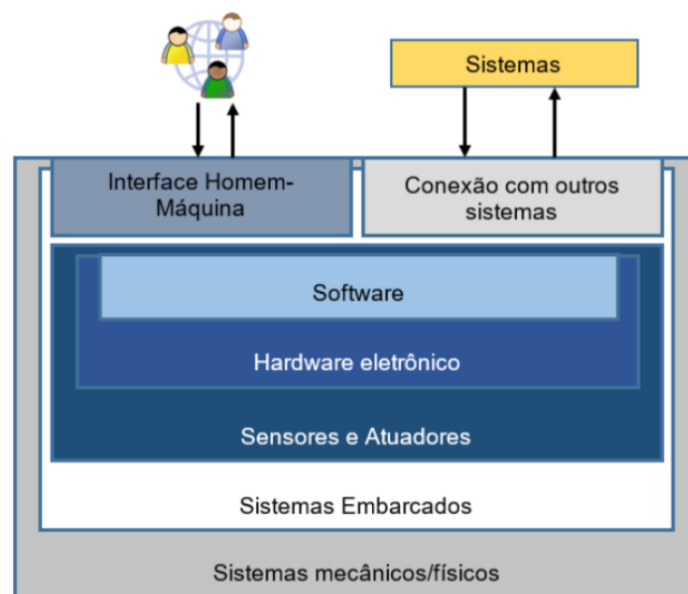
O termo *Cyber-Physical Systems* (CPS), ou Sistemas Ciber-Físicos, foi proposto em 2006 por um grupo de pesquisadores ligados à *National Science Foundation* (NSF) dos Estados Unidos, durante o primeiro workshop da área realizado em Austin, Texas, com o propósito de descrever o acoplamento crescente entre dispositivos de controle, computação e redes de comunicação (Pisching, 2017; Monostori *et al.*, 2016). Monostori *et al.* (2016) definem CPS como sistemas de entidades computacionais colaborativas em conexão intensiva com o mundo físico circundante e com seus processos em curso, fornecendo e consumindo, ao mesmo tempo, serviços de acesso e processamento de dados disponíveis na internet. Mais relevante que a união das partes, observam os autores, é a *interseção* entre o mundo físico e o cibernético: compreender as componentes separadamente não é suficiente; é a interação entre elas o que caracteriza o CPS.

Os CPS herdam fortemente o legado dos *sistemas embarcados* (sistemas computacionais integrados a sistemas mecânicos ou elétricos para executar funções específicas com restrições de tempo real), mas ampliam esse escopo ao incorporar conectividade em rede, autonomia

e cooperação entre dispositivos (Monostori *et al.*, 2016). No domínio da manufatura, são frequentemente denominados *Cyber-Physical Production Systems* (CPPS) ou Sistemas Ciber-Físicos de Produção, sem que isso represente diferença conceitual significativa: trata-se de CPS contextualizados em maquinários, sistemas de estoque, sistemas de distribuição e linhas produtivas, com capacidade de operar de forma autônoma, trocar informações, disparar eventos e controlar uns aos outros independentemente (Pisching, 2017).

A Figura 9 ilustra a estrutura básica de um CPS, organizada em torno do acoplamento entre sistemas físicos e a camada computacional. Sensores e atuadores constituem a interface bidirecional com o processo físico; o *hardware* eletrônico e os sistemas embarcados executam algoritmos de aquisição, controle e diagnóstico em tempo real; e a camada de *software*, conectada a outros sistemas e à internet, viabiliza serviços de mais alto nível, como otimização, manutenção preditiva e cooperação multi-agente.

Figura 9: Estrutura e componentes básicos de um CPS



Fonte: Pisching (2017, p. 44).

Monostori *et al.* (2016) destacam três características essenciais dos CPS aplicados à manufatura. A primeira é a **inteligência** (*smartness*): os elementos do sistema são capazes de adquirir informações de seu entorno e agir autonomamente, sem depender de comando externo contínuo. A segunda é a **conectividade** (*connectedness*): cada elemento estabelece e utiliza conexões com os demais elementos do sistema, com seres humanos e com serviços e conhecimento disponíveis na internet, viabilizando cooperação e colaboração. A terceira é a **responsividade** (*responsiveness*): o sistema reage, em tempo apropriado, a mudanças internas e externas, ajustando seu comportamento sem necessidade de reconfiguração manual. Esse conjunto de capacidades também sustenta expectativas mais amplas: auto-organização, automanutenção, autorreparo, transparência, previsibilidade e diagnóstico remoto.

No plano arquitetural, os CPS rompem parcialmente com a tradicional pirâmide de automação descrita na Subseção 2.5. Os níveis de controle e campo, com CLPs próximos aos processos técnicos e restrições de tempo real, permanecem em grande medida; contudo, nos níveis superiores, observa-se um modo de funcionamento mais descentralizado, no qual decisões antes concentradas em estações de supervisão passam a ser distribuídas entre dispositivos

colaborativos (Monostori *et al.*, 2016). Como observa Pisching (2017), os CPS conectados via internet podem ser compreendidos como elementos da IoT, isto é, os CPS oferecem a base física e computacional sobre a qual a IoT, conjugada à Internet dos Serviços e dos Dados, viabiliza o conceito de Indústria 4.0.

2.8 Indústria 4.0

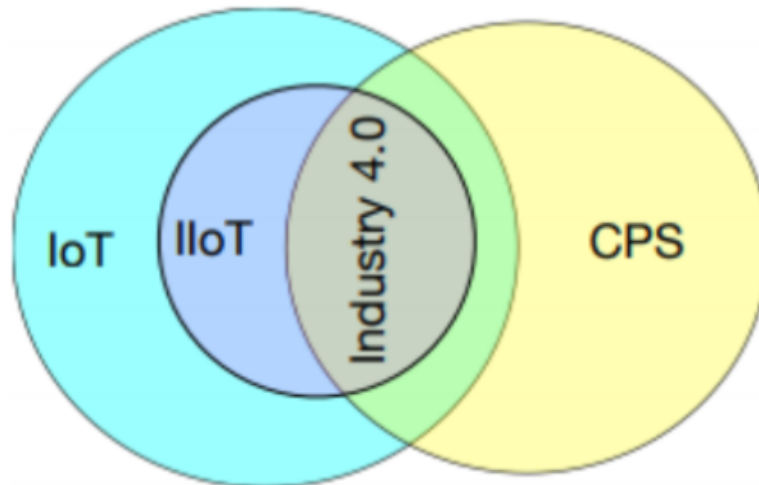
O termo *Industrie 4.0* foi apresentado pela primeira vez em 2011, na Feira de Hannover, e oficializado em abril de 2013, no âmbito da estratégia de alta tecnologia do governo alemão, como parte de um plano de ação para elevar o nível de automatização e digitalização dos sistemas produtivos do país (Pisching, 2017; Bigheti, 2020). A nomenclatura, em referência ao versionamento de *software*, posiciona-se como o anúncio *ex-ante* de uma quarta revolução industrial, somando-se às transformações anteriores: a primeira, no final do século XVIII, baseada na mecanização movida a vapor e água; a segunda, no início do século XX, marcada pela eletricidade e pela produção em massa; e a terceira, na segunda metade do século XX, caracterizada pela eletrônica e pelas tecnologias da informação aplicadas à automação (Lasi *et al.*, 2014; Pisching, 2017). No Brasil, o conceito tornou-se conhecido também como Manufatura Avançada, com discussões intensificadas a partir de 2013; nos Estados Unidos, recebe a denominação *Smart Industry* (Bigheti, 2020).

Lasi *et al.* (2014) caracterizam a Indústria 4.0 como o resultado de duas forças simultâneas. De um lado, há um forte *application-pull* associado a mudanças sociais, econômicas e políticas: períodos de desenvolvimento mais curtos (*time to market*), individualização da demanda, chegando ao limite teórico do lote unitário, maior flexibilidade na produção, descentralização das decisões e maior eficiência no uso de recursos. De outro lado, há um intenso *technology-push* fundado em três tendências tecnológicas convergentes: a contínua mecanização e automação dos processos, a digitalização e o aumento da conectividade entre componentes técnicos, e a miniaturização do *hardware*, que permite incorporar capacidade computacional onde antes isso seria inviável.

Bigheti (2020) sintetiza esse contexto definindo a Indústria 4.0 como um conjunto de tecnologias aplicadas ao contexto industrial e coordenadas para conferir competitividade, otimizar a cadeia produtiva, agregar valor ao produto, racionalizar o uso de recursos e customizar soluções. Entre as tecnologias habilitadoras citadas pelo autor destacam-se: o IPv6, que amplia o espaço de endereçamento a todos os equipamentos; as redes sem fio para flexibilização da aquisição de dados; os sistemas ciber-físicos (CPS) e os sistemas de controle em rede; o uso de identificadores RFID para rastreamento; a virtualização de sistemas e serviços; a IIoT e a computação em nuvem; e o uso massivo de dados (*big data*) para apoio à decisão. O conceito-síntese, em diversas formulações, é o de *fábrica inteligente* (*smart factory*): uma planta na qual produtos e máquinas comunicam-se por meio de CPS e IoT ao longo do ciclo produtivo, em um ambiente apoiado por infraestrutura de nuvem que combina Internet dos Serviços e Internet dos Dados (Pisching, 2017; Lasi *et al.*, 2014).

A Figura 10 sintetiza graficamente as relações entre os quatro conceitos discutidos até aqui. A IoT figura como o universo mais amplo, abrangendo objetos conectados em contextos de consumo e industriais; a IIoT corresponde à interseção da IoT com o domínio industrial; os CPS situam-se como categoria parcialmente distinta, caracterizada pelo forte acoplamento computação-físico em tempo real; e a Indústria 4.0 emerge na interseção entre IIoT e CPS, materializando o conceito de fábrica inteligente sob infraestrutura de nuvem.

Figura 10: Interseções de IoT, CPS, IIoT e Indústria 4.0

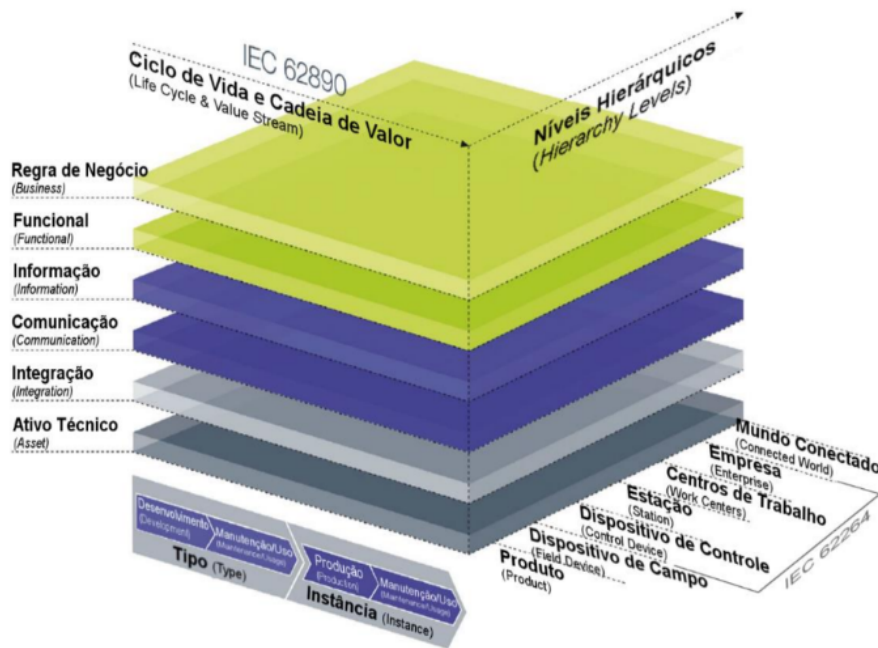


Fonte: Bigheti (2020, p. 27).

No nível de princípios de projeto, a Indústria 4.0 é frequentemente descrita por seis pilares, segundo Pisching (2017): **interoperabilidade**, capacidade de máquinas, dispositivos, sensores e pessoas se comunicarem por meio da IoT; **virtualização**, criação de cópias digitais do mundo físico que permitem simulação e supervisão; **descentralização**, distribuição da capacidade de decisão pelos diversos pontos do sistema; **operação em tempo real**, com coleta e análise contínuas de dados; **orientação a serviços**, em que funcionalidades são expostas e compostas via redes; e **modularidade**, permitindo adaptação flexível a mudanças de demanda e a novos cenários. Esses princípios materializam-se nos três aspectos centrais identificados por Bange-mann et al. (2016): uma nova forma de organizar e controlar a cadeia de valor ao longo de todo o ciclo de vida dos produtos; a disponibilidade de informações relevantes em tempo real, em todas as instâncias envolvidas; e a constituição de redes de valor entre empresas, dinâmicas e auto-organizáveis, suportadas pela interconexão de pessoas, “coisas” e sistemas (Pisching, 2017).

Para organizar e padronizar a transição em torno desses princípios, foi apresentado em 2015, também na Feira de Hannover, o *Reference Architectural Model for Industry 4.0* (RAMI 4.0). Trata-se de um modelo tridimensional que compatibiliza, em um único arcabouço, normas vigentes do setor produtivo, como a ISO/IEC 62264 (integração entre sistemas corporativos e de controle), a IEC 62541 (OPC-UA), a IEC 61512 (controle em batelada) e a IEC 62890 (ciclo de vida), com a perspectiva de integração vertical (entre níveis hierárquicos), horizontal (entre empresas em uma cadeia de valor) e fim a fim (entre fases de engenharia) (Pisching, 2017; Bigheti, 2020). A Figura 11 ilustra a estrutura tridimensional do modelo.

Figura 11: Modelo de Arquitetura de Referência para a Indústria 4.0 (RAMI 4.0)



Fonte: Pisching (2017, p. 76).

Por fim, a Indústria 4.0 amplia o escopo dos sistemas SCADA tradicionais discutidos na Subseção anterior. Se historicamente o SCADA atuava como um sistema fechado de supervisão e aquisição de dados, ligado a CLPs por barramentos proprietários e a uma sala de controle local, a perspectiva da Indústria 4.0 o reposiciona como um nó de uma rede de serviços muito mais ampla, que integra produção, manutenção, logística, vendas e clientes em uma cadeia de valor digital (Bigheti, 2020; Bangemann *et al.*, 2014). Esse reposicionamento é o pano de fundo conceitual para a proposta de sistema supervisório baseado em nuvem e orientado a serviços apresentada no restante deste trabalho.

2.9 Computação em Nuvem

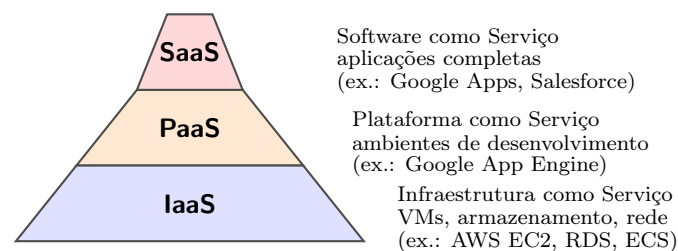
O termo *Cloud Computing*, ou Computação em Nuvem, surgiu em 2006, em uma palestra de Eric Schmidt, da Google, sobre o modelo de gerenciamento dos *data centers* da empresa (Henriques da Silva, 2010). Desde então, consolidou-se como um paradigma em que dados e aplicações dos usuários são movidos para grandes centros de armazenamento, e a infraestrutura computacional passa a ser distribuída como um conjunto de serviços disponibilizados pela internet. A “nuvem” é, em essência, uma camada conceitual que esconde a infraestrutura e os recursos físicos, expondo ao usuário uma interface padronizada por meio da qual uma infinidade de serviços é oferecida (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011). Trata-se de uma evolução natural dos modelos de computação sob demanda, com forte parentesco com o conceito de *Utility Computing*: assim como a energia elétrica e o fornecimento de água, a capacidade computacional passa a ser tratada como uma utilidade tarifada pelo consumo.

Para que uma plataforma seja considerada efetivamente uma solução de computação em nuvem, ela deve atender a cinco características essenciais amplamente reproduzidas na literatura (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011): **self-service sob demanda**, em que o usuário adquire unilateralmente recursos computacionais sem interação humana com o

provedor; **amplo acesso à rede**, com recursos acessíveis por meio de mecanismos padronizados a partir de plataformas heterogêneas (navegadores, *smartphones* e dispositivos industriais); **pooling de recursos**, em um modelo multi-inquilino (*multi-tenant*) que aloca dinamicamente recursos físicos e virtuais a múltiplos consumidores; **elasticidade rápida**, com capacidade escalando automaticamente conforme a demanda e criando para o usuário a ilusão de recursos ilimitados; e **serviço medido**, com uso monitorado, controlado e tarifado de forma transparente, frequentemente vinculado a acordos de nível de serviço (*Service Level Agreement*, SLA).

A arquitetura da computação em nuvem é tradicionalmente hierarquizada em três camadas de serviço, comumente representadas em forma de pirâmide (Figura 12). A base é a **Infraestrutura como Serviço** (IaaS, *Infrastructure as a Service*), que disponibiliza recursos virtualizados de *hardware* como máquinas virtuais, armazenamento, redes e roteadores, além de uma interface única para administração da infraestrutura (Pedrosa; Nogueira, 2011). A camada intermediária é a **Plataforma como Serviço** (PaaS, *Platform as a Service*), que oferece ambientes prontos para o desenvolvimento e implantação de aplicações, com sistemas operacionais, linguagens de programação e ambientes de desenvolvimento pré-configurados, dispensando o desenvolvedor de provisionar manualmente processadores e memória (Henriques da Silva, 2010). A camada superior é o **Software como Serviço** (SaaS, *Software as a Service*), na qual aplicações completas são oferecidas ao usuário final por meio de portais web, executando inteiramente na nuvem do provedor; o modelo SaaS é discutido em profundidade na Subseção 2.19.

Figura 12: Modelos de serviço em computação em nuvem (pirâmide IaaS, PaaS, SaaS)



Fonte: Elaborado pelo autor com base em Henriques da Silva (2010) e Pedrosa e Nogueira (2011).

No que diz respeito ao modelo de implantação, quatro variantes são reconhecidas (Pedrosa; Nogueira, 2011; Henriques da Silva, 2010): a **nuvem pública**, operada por um terceiro e disponibilizada ao público em geral ou a grandes grupos industriais; a **nuvem privada**, construída exclusivamente para uma organização, com infraestrutura sob seu controle; a **nuvem comunitária**, compartilhada por um grupo de organizações com requisitos e políticas semelhantes; e a **nuvem híbrida**, combinação dos modelos anteriores, em que a infraestrutura privada pode ser ampliada por reserva de recursos em uma nuvem pública, também denominada “computação em ondas” por permitir absorver picos de demanda sem comprometer o nível de serviço.

Três papéis principais participam de uma solução em nuvem (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011). O **provedor de serviço** é responsável por disponibilizar, gerenciar e monitorar toda a infraestrutura, garantindo desempenho e segurança. O **desenvolvedor** constrói aplicações e serviços sobre a infraestrutura disponibilizada pelo provedor. O **usuário final** é quem consome os recursos oferecidos. No contexto deste trabalho, o autor atua simultaneamente como desenvolvedor (construindo o sistema supervisor) e como contratante de um provedor comercial, a Amazon Web Services (AWS), discutida em detalhe na Subseção 2.15.

As vantagens comumente atribuídas ao modelo incluem mobilidade (acesso a dados e aplicações de qualquer local com conexão à internet), pagamento por uso (que evita desperdício de recursos), escalabilidade rápida, confiabilidade dos grandes provedores e compartilhamento eficiente de recursos (Pedrosa; Nogueira, 2011). Em contrapartida, ainda persistem desafios significativos: **segurança**, dado que os dados saem do controle local e ficam expostos a ataques externos; **escalabilidade**, que exige aplicações projetadas para tirar proveito da elasticidade; **interoperabilidade**, com portabilidade entre provedores ainda limitada; **confiabilidade**, dado que sistemas complexos podem falhar; e **disponibilidade**, lembrando que mesmo grandes serviços como o Gmail já apresentaram períodos fora do ar (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011).

No contexto da Indústria 4.0 discutido na Subseção 2.8, a computação em nuvem é a infraestrutura por excelência que viabiliza a integração de produtos, máquinas, sensores e serviços ao longo de toda a cadeia de valor (Bigheti, 2020; Pisching, 2017). Sistemas SCADA tradicionais, antes confinados a salas de controle locais e ligados a CLPs por barramentos proprietários, são reposicionados como nós de uma rede distribuída de serviços. A aquisição de dados pode ocorrer remotamente em centenas de pontos de medição, a persistência histórica é delegada a bancos gerenciados na nuvem, e a supervisão é acessada via navegador web em qualquer ponto da rede. Essa é exatamente a arquitetura adotada pelo sistema supervisório proposto neste trabalho, cujos serviços executam em uma infraestrutura híbrida (*gateways* e modems no campo, serviços em nuvem na infraestrutura central). Aspectos de virtualização que sustentam essa arquitetura são detalhados em Salvador (2019b).

2.10 Banco de Dados

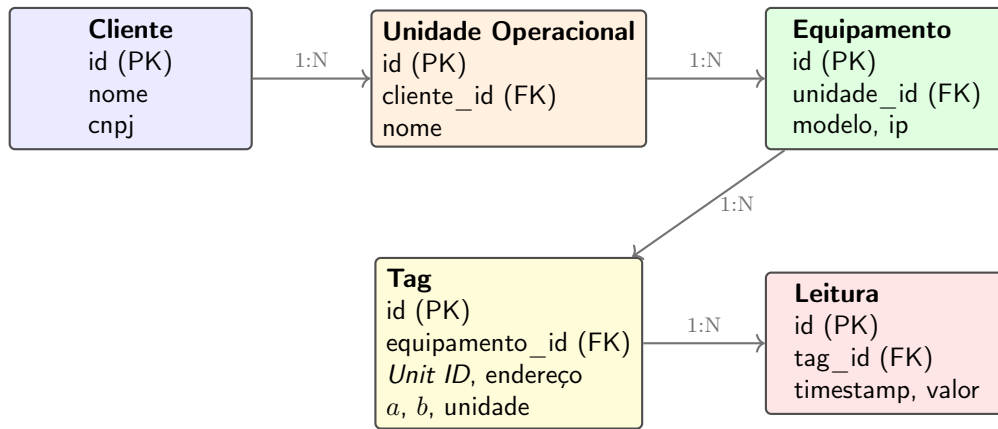
Em qualquer sistema supervisório, o banco de dados é o componente responsável pela persistência das informações que descrevem o cadastro do sistema (unidades operacionais, equipamentos, *tags*, parâmetros de comunicação, usuários, alarmes) e pelo armazenamento histórico das leituras adquiridas em campo. Diferentes formas de banco de dados são adequadas a propósitos diferentes: bancos relacionais favorecem o cadastro estrutural e as relações entre entidades; bancos orientados a séries temporais favorecem o armazenamento massivo de leituras com carimbo de tempo; bancos NoSQL (documentais, chave-valor, grafos) favorecem cenários de alta variabilidade de esquema. Boas práticas de modelagem industrial recomendam estruturação hierárquica baseada em padrões como ISA-95 e modelo de referência Purdue, traduzidos para o modelo relacional por meio de tabelas e chaves estrangeiras (Silva, 2023).

O **modelo relacional**, proposto por Edgar F. Codd em 1970 e materializado em bancos como PostgreSQL, MySQL, Oracle e SQL Server, organiza os dados em **tabelas** compostas por linhas (registros) e colunas (atributos). Cada tabela é definida por um **esquema** (estrutura de colunas com seus tipos), uma **chave primária** (que identifica unicamente cada linha) e, opcionalmente, **chaves estrangeiras** (que estabelecem relações com outras tabelas). A linguagem de consulta padrão é o SQL (*Structured Query Language*), com operações como **SELECT**, **INSERT**, **UPDATE**, **DELETE** e suporte a **JOIN**, agregações, transações e índices.

Para o sistema supervisório proposto, o modelo relacional foi escolhido por três razões. Primeiro, a hierarquia natural do cadastro (cliente, unidade operacional, equipamento, *tag*) modela-se bem com chaves estrangeiras. Segundo, a integridade referencial garantida pelo banco evita classes inteiras de inconsistências (uma *tag* não pode existir sem um equipamento associado, por exemplo). Terceiro, o ecossistema de bibliotecas para Node.js, linguagem do *backend*, tem ampla maturidade com SQL. A escolha específica recaiu sobre o PostgreSQL, banco relacional de código aberto com forte suporte a tipos complexos (JSON, *arrays*, dados

geométricos), consultas em janelas de tempo e extensões para séries temporais, gerenciado em produção pelo serviço Amazon RDS (Subseção 2.15).

Figura 13: Trecho simplificado do esquema relacional do sistema supervisorio



Fonte: Autor.

A Figura 13 ilustra um trecho simplificado do esquema relacional do sistema, com as principais entidades e suas cardinalidades: cada *Cliente* pode ter várias *Unidades Operacionais*; cada unidade tem vários *Equipamentos*; cada equipamento expõe várias *Tags*; cada *tag* gera várias *Leituras* no histórico, cada uma com seu carimbo de tempo e valor convertido em unidade de engenharia (conforme procedimento descrito na Subseção 2.3). Os coeficientes *a* e *b* do ajuste linear são armazenados como atributos da própria *tag*, permitindo o cálculo da unidade de engenharia diretamente pela camada de aquisição no momento da leitura.

Boas práticas de uso do banco em ambiente de produção incluem: indexação cuidadosa dos campos mais consultados (especialmente *tag_id* e *timestamp* na tabela de *Leituras*); particionamento por intervalo de tempo dos históricos para evitar tabelas monolíticas; execução assíncrona de operações pesadas para não bloquear o ciclo de aquisição; e definição de filtros temporais curtos nas consultas históricas para evitar travamentos da interface (Damin, 2022).

A redundância e a alta disponibilidade do banco são tratadas em nível de infraestrutura por meio dos recursos do provedor de nuvem: réplicas em zonas de disponibilidade distintas, *snapshots* automatizados para recuperação ponto-a-ponto e *failover* transparente para a aplicação (Salvador, 2019a,b). Essas garantias, antes onerosas em infraestrutura local, tornam-se opções de configuração no console do serviço gerenciado.

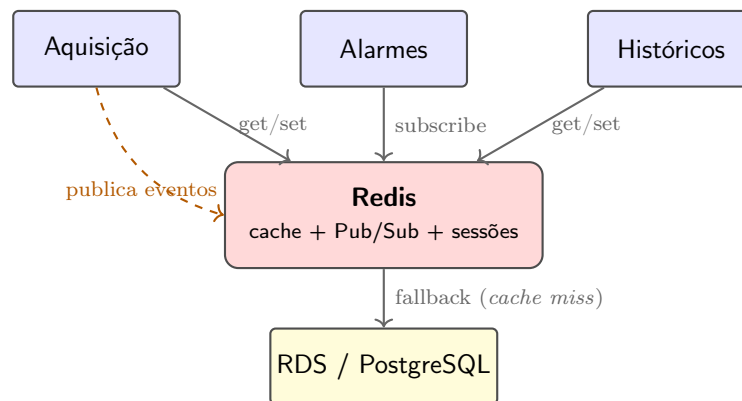
2.11 Redis

O **Redis** (*Remote Dictionary Server*) é um armazenamento de dados em memória, de código aberto, que opera como banco chave-valor de altíssimo desempenho. Diferentemente dos bancos relacionais discutidos na Subseção 2.10, em que a persistência em disco é a prioridade arquitetural, o Redis mantém todo o conjunto de trabalho em memória RAM, oferecendo latências sub-milissegundo para operações de leitura e escrita. Essa característica torna-o especialmente adequado para cenários em que velocidade de acesso é crítica e em que a perda eventual de dados pode ser tolerada ou compensada por mecanismos auxiliares de persistência.

Os tipos de dados oferecidos pelo Redis vão além do simples par chave-valor textual: a biblioteca suporta nativamente *strings*, listas, conjuntos, conjuntos ordenados (*sorted sets*),

hashes, *bitmaps*, *streams* e tipos geoespaciais. Cada tipo vem com operações otimizadas em complexidade computacional (frequentemente $O(1)$ ou $O(\log n)$), o que permite usar o Redis não apenas como cache, mas como estrutura algorítmica auxiliar para problemas como contagem de eventos, filas de prioridade, classificações em tempo real (*leaderboards*), expiração automática de chaves (*time-to-live*, TTL) e travas distribuídas. Para garantir persistência opcional, o Redis oferece dois mecanismos: *snapshots* periódicos do estado em memória (RDB) e um log de operações de escrita (AOF, *Append-Only File*), configuráveis independentemente conforme o compromisso desejado entre desempenho e durabilidade.

Figura 14: Papéis do Redis no sistema supervisorio: cache de leituras quentes e barramento de eventos



Fonte: Autor.

Em arquiteturas distribuídas como a deste trabalho, o Redis cumpre tipicamente três papéis principais. O primeiro é o de **cache de leituras frequentes**: valores acessados muitas vezes ao longo de janelas curtas (estado atual de cada *tag*, configuração de equipamentos, perfil de usuário autenticado) são mantidos em memória com TTL apropriado, evitando consultas repetidas ao banco relacional e reduzindo a latência percebida pelos usuários. O segundo é o de **barramento de mensagens publicador-assinante** (Pub/Sub) entre microserviços (Subseção 2.17): o serviço de aquisição publica eventos em canais nomeados, e os serviços de alarmes, históricos e *frontend* ao vivo se inscrevem para receber os valores em tempo real, sem necessidade de *polling*. O terceiro é o de **armazenamento de estado efêmero**: tokens de autenticação, contadores de *rate limiting* e travas distribuídas que evitam processamento duplicado de comandos críticos.

A combinação de Redis com bancos persistentes constitui um padrão arquitetural conhecido como *cache-aside* ou *read-through*: o serviço primeiro consulta o Redis; em caso de *cache miss*, busca no banco persistente e popula o cache para os próximos acessos. Esse padrão aproveita o princípio de localidade temporal (valores recém-acessados tendem a ser acessados de novo em breve) sem comprometer a fonte de verdade (o banco relacional), e é especialmente eficaz em sistemas de supervisão, em que poucos pontos críticos concentram a maior parte das consultas de operadores e do motor de alarmes.

Para garantir alta disponibilidade em produção, o Redis oferece duas estratégias complementares. O **Redis Sentinel** monitora uma instância primária e suas réplicas, promovendo automaticamente uma réplica em caso de falha do nó principal. O **Redis Cluster** distribui as chaves entre múltiplos nós usando *hashing* consistente, permitindo escala horizontal de capacidade e taxa de transferência, com tolerância a falhas em nível de partição. Em ambientes em

nuvem, esses mecanismos são frequentemente oferecidos como recursos do serviço gerenciado, dispensando a configuração manual.

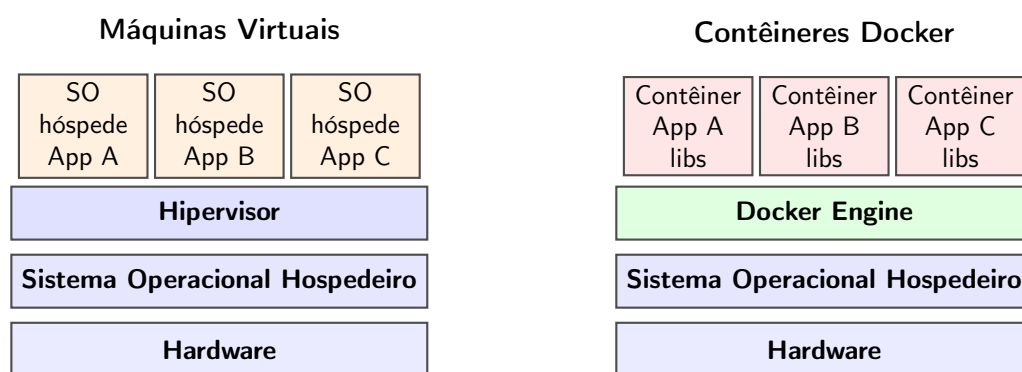
No sistema supervisório proposto, o Redis é provisionado como serviço gerenciado por meio do Amazon ElastiCache (extensão da AWS discutida na Subseção 2.15), com replicação em zonas de disponibilidade distintas para alta disponibilidade. O cluster atende simultaneamente às três funções descritas e ilustradas na Figura 14: cache das últimas leituras de cada *tag* (lidas pelo *frontend* a cada atualização de tela), canal Pub/Sub para eventos de aquisição (consumidos pelos serviços de alarmes e históricos) e armazenamento de sessões da camada de autenticação. Esse uso multifacetado é típico do ecossistema de microsserviços contemporâneo e está alinhado com as escolhas arquiteturais discutidas por Bigheti (2020) para sistemas industriais orientados a serviços.

2.12 Docker

O Docker é uma plataforma de virtualização baseada em **contêineres**, unidades de empacotamento de software que reúnem código, bibliotecas, configurações e dependências em um único artefato portátil. Diferentemente das máquinas virtuais tradicionais discutidas em Salvador (2019b), que emulam uma máquina inteira (incluindo o sistema operacional), os contêineres compartilham o *kernel* do sistema hospedeiro e isolam apenas o espaço de processos e o sistema de arquivos da aplicação. Essa diferença arquitetural torna os contêineres ordens de magnitude mais leves: enquanto uma VM pode levar minutos para iniciar e ocupar gigabytes em disco, um contêiner inicia em segundos e consome dezenas a centenas de megabytes.

O modelo conceitual do Docker apoia-se em três artefatos principais. O **Dockerfile** é um arquivo de texto que descreve, instrução por instrução, como construir a imagem (sistema operacional base, instalação de dependências, cópia de código, comandos de inicialização). A **imagem** é o resultado da construção, um sistema de arquivos somente leitura organizado em camadas que pode ser versionado e armazenado em registros (*registries*) como Docker Hub ou Amazon ECR (*Elastic Container Registry*). O **contêiner** é uma instância em execução de uma imagem, com seu próprio espaço de processos, rede e armazenamento volátil.

Figura 15: Arquitetura comparada: máquinas virtuais (esquerda) e contêineres Docker (direita)



Fonte: Elaborado pelo autor com base em Salvador (2019b).

A adoção de contêineres traz vantagens diretas para sistemas distribuídos modernos, particularmente os baseados em microsserviços (ver Subseção 2.18). A primeira é a **reprodutibilidade do ambiente**: o mesmo Dockerfile gera a mesma imagem em desenvolvimento, em homologação e em produção, eliminando a clássica frase “funciona na minha máquina”. A

segunda é o **isolamento de dependências**: dois serviços podem usar versões diferentes da mesma biblioteca sem conflito, cada um em seu contêiner. A terceira é a **portabilidade**: a imagem construída uma vez pode ser implantada em qualquer ambiente que execute o Docker Engine (estação de trabalho local, servidor on-premises, AWS, Google Cloud, Azure).

A composição de múltiplos contêineres relacionados é geralmente expressa em um arquivo `docker-compose.yml`, que descreve os serviços, suas variáveis de ambiente, volumes persistentes e redes virtuais que os conectam. Em ambientes de produção em larga escala, orquestradores como Kubernetes ou serviços gerenciados como Amazon ECS (*Elastic Container Service*, ver Subseção 2.15) assumem o papel de provisionar instâncias dos contêineres, monitorá-las, reiniciá-las em caso de falha e escalá-las horizontalmente conforme a carga.

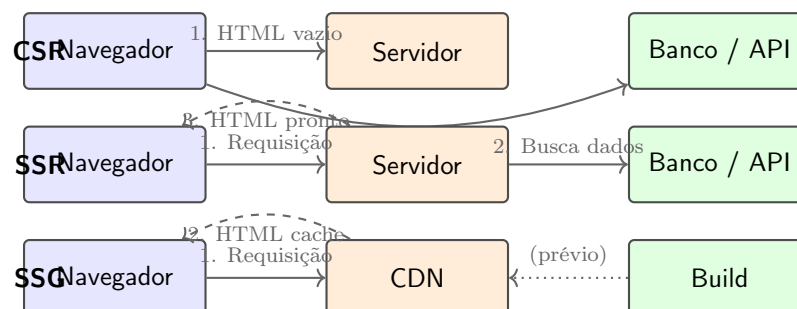
No sistema supervisorio proposto, cada serviço de domínio (aquisição, configuração, alarmes, históricos, autenticação) é empacotado como um contêiner Docker independente. Essa decisão de projeto está alinhada com a tendência de Bigheti (2020) de adotar arquiteturas orientadas a microsserviços para a Indústria 4.0, e viabiliza implantação independente, escalabilidade por serviço e ciclo de manutenção desacoplado entre as partes do sistema.

2.13 Next.js

O Next.js é um *framework* de desenvolvimento web construído sobre o React, biblioteca JavaScript criada pela Meta (anteriormente Facebook) para a construção de interfaces de usuário baseadas em componentes. Enquanto o React, por si só, é uma biblioteca de renderização no lado do cliente (*client-side rendering*, CSR), o Next.js estende-o com capacidades de renderização no servidor (*server-side rendering*, SSR), geração estática (*static site generation*, SSG) e renderização incremental, suprimindo lacunas importantes para aplicações de produção que exigem desempenho, otimização de mecanismos de busca (SEO) e tempo de carregamento inicial reduzido.

O conceito-chave para entender o Next.js é a separação entre três estratégias de renderização. No **CSR**, o navegador recebe um arquivo HTML mínimo e um *bundle* JavaScript pesado, e toda a construção da interface ocorre no cliente; é o modelo de aplicações de página única (SPA) tradicionais. No **SSR**, o servidor renderiza a página HTML completa a cada requisição e envia ao navegador um conteúdo já pronto, melhorando a percepção de carregamento e a indexação por buscadores. No **SSG**, a página é renderizada uma única vez no momento do *build* e servida como HTML estático para todos os usuários, oferecendo desempenho máximo para conteúdo que muda pouco. O Next.js permite escolher a estratégia rota a rota, conforme as características de cada tela do sistema.

Figura 16: Estratégias de renderização do Next.js: CSR, SSR e SSG



Fonte: Autor.

Outro elemento estruturante do Next.js é o **roteamento baseado em sistema de arquivos**: cada arquivo dentro do diretório `app/` ou `pages/` torna-se automaticamente uma rota da aplicação, com convenções para rotas dinâmicas, aninhamento e layouts compartilhados. Esse modelo elimina a necessidade de configurar manualmente um roteador, reduz o acoplamento entre estrutura de pastas e estrutura de URLs, e facilita a manutenção em projetos grandes.

O Next.js oferece ainda otimizações nativas relevantes para aplicações de tempo real, como o sistema de supervisão proposto: *code splitting* automático (cada rota carrega apenas o JavaScript necessário), *prefetching* inteligente de páginas vinculadas no campo de visão do usuário, otimização automática de imagens, suporte nativo a TypeScript, e camada de funções de servidor (*Server Actions*) que dispensa a criação manual de rotas de API para operações triviais.

No sistema supervisorio, o Next.js é a base do *frontend* de supervisão, composto por telas de visualização de pontos em tempo real, dashboards de tendências históricas, tela de cadastro de equipamentos e *tags*, painéis de alarmes e formulários administrativos. As telas mais sensíveis a desempenho (visualização ao vivo) usam renderização no cliente com atualização incremental por WebSocket; as telas de cadastro e relatórios usam SSR, beneficiando-se do pré-processamento e da autenticação no servidor; e a documentação e as páginas públicas usam SSG.

2.14 Fastify

O Fastify é um *framework* web para Node.js, projetado com foco em desempenho, baixa sobrecarga e ergonomia para construir APIs HTTP. Posiciona-se como uma alternativa moderna ao Express.js (durante muitos anos o *de facto* para Node.js), com benchmarks consistentes de duas a três vezes mais requisições por segundo no mesmo *hardware*, graças a um sistema de roteamento otimizado, serialização de respostas baseada em JSON Schema e um ciclo de processamento de requisições estritamente desacoplado.

A unidade fundamental de organização do código no Fastify é o **plugin**. Cada plugin encapsula um conjunto de rotas, validações, decoradores e hooks que podem ser carregados isoladamente, com escopo próprio. Essa arquitetura favorece o desenvolvimento modular: um plugin de autenticação, outro de *logging*, outro de conexão com o banco de dados e outro com as rotas de domínio podem ser compostos em uma única aplicação sem acoplamento direto, e podem ser testados em isolamento. A inicialização da aplicação é declarativa, com a ordem de carregamento explicitada por meio de chamadas `fastify.register(...)`.

O Fastify também adota o **JSON Schema** como mecanismo padrão para validação de entrada e definição de contratos. A cada rota é possível associar um esquema que descreve o formato esperado de *params*, *query string*, *body* e resposta. Esse esquema é compilado em tempo de inicialização para funções altamente otimizadas, oferecendo ganho duplo: validação rápida das requisições e serialização rápida das respostas, evitando a sobrecarga do `JSON.stringify` genérico do Node.js. Adicionalmente, o esquema serve como fonte única para geração automática de documentação OpenAPI (Swagger), reduzindo a divergência típica entre código e documentação.

Outros recursos relevantes do Fastify para o cenário deste trabalho incluem: *logging* estruturado por padrão (com Pino, biblioteca de *logging* de altíssimo desempenho); suporte nativo a *lifecycle hooks* (`onRequest`, `preHandler`, `onSend`, `onResponse`) que permitem interceptar a requisição em pontos bem definidos do ciclo; suporte a TypeScript com inferência de tipos a partir do esquema JSON; e integração natural com filas (Bull, BullMQ) e cache (Redis) por meio de plugins maduros do ecossistema.

No sistema supervisorio, o Fastify é a fundação dos serviços HTTP do *backend*: cada mi-

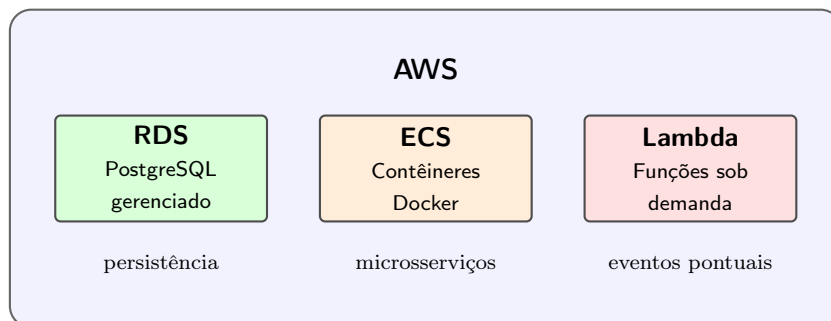
crossserviço (aquisição, configuração, alarmes, históricos, autenticação) expõe sua API REST por meio de uma instância de Fastify, com rotas, esquemas e *hooks* próprios. A comunicação entre os microsserviços, quando síncrona, ocorre via Fastify; quando assíncrona, ocorre via barramento de mensagens do Moleculer (Subseção 2.18).

2.15 Amazon Web Services (AWS)

A Amazon Web Services (AWS) é o serviço de computação em nuvem da Amazon e atualmente o maior provedor comercial do segmento. Originalmente lançada em 2006 (mesmo ano do nascimento do termo *Cloud Computing* segundo Henriques da Silva (2010)), a plataforma oferece, hoje, mais de duzentos serviços que cobrem todas as camadas do modelo apresentado na Subseção 2.9, desde IaaS de baixo nível (máquinas virtuais EC2, redes virtuais VPC, armazenamento S3) até SaaS gerenciados (mensageria, autenticação, serviços de pagamento, aprendizado de máquina). A precificação segue o modelo *pay-per-use* característico da nuvem (Pedrosa; Nogueira, 2011): cobra-se por hora ou segundo de uso, por gigabyte armazenado, por requisição processada, e a fatura mensal espelha o consumo real.

Para o sistema supervisorio proposto, três serviços específicos da AWS são adotados (Figura 17): o **RDS** (*Relational Database Service*) para o banco de dados PostgreSQL gerenciado; o **ECS** (*Elastic Container Service*) para a orquestração dos contêineres Docker dos microsserviços; e o **Lambda** para a execução de funções pontuais sem provisionamento de servidor. Esses três serviços, em conjunto, materializam uma arquitetura conhecida como *serverless-first*, em que a maior parte da complexidade operacional (atualização de sistema operacional, balanceamento de carga, *failover*, *scaling*) é delegada ao provedor.

Figura 17: Serviços AWS empregados no sistema supervisorio (RDS, ECS, Lambda)



Fonte: Autor.

O **Amazon RDS** é um serviço gerenciado de banco de dados relacional que suporta múltiplas *engines* (PostgreSQL, MySQL, MariaDB, Oracle, SQL Server e o Amazon Aurora). O provedor assume responsabilidades historicamente trabalhosas: aplicação de patches de segurança, *backups* automáticos diários, criação de réplicas para alta disponibilidade em zonas de disponibilidade distintas, *failover* automático em caso de falha da instância primária, e monitoramento contínuo de métricas operacionais. Como discutido por Salvador (2019a), esses mecanismos atingem, em ambiente de nuvem, níveis de confiabilidade comparáveis aos de soluções dedicadas, sem o custo capital de infraestrutura local.

O **Amazon ECS** é um serviço de orquestração de contêineres Docker (ver Subseção 2.12). A cada serviço do sistema corresponde uma *task definition*, que descreve a imagem Docker, os limites de CPU e memória, as variáveis de ambiente e as portas expostas. O ECS é responsável

por executar essas *tasks* em uma frota de instâncias EC2 (ou em modo *serverless* via Fargate, em que a Amazon gerencia também a camada de máquina virtual), por monitorá-las, reiniciá-las quando falham, distribuí-las em zonas de disponibilidade e escalá-las horizontalmente conforme métricas de utilização.

O **AWS Lambda** introduz o paradigma *Functions as a Service* (FaaS): blocos de código sem servidor (*serverless*) que executam em resposta a eventos (chegada de uma mensagem em uma fila, alteração em um *bucket* S3, requisição em um *endpoint* HTTP, expiração de um *timer*). O usuário envia apenas o código da função; a Amazon provisiona o ambiente de execução, gerencia a escalabilidade e cobra por milissegundo efetivamente utilizado. No sistema supervisorio, o Lambda é adequado para tarefas pontuais como geração de relatórios sob demanda, envio de notificações via email ou SMS quando um alarme é disparado, e rotinas periódicas de manutenção (limpeza de logs, arquivamento de históricos antigos) que não justificariam um contêiner sempre em execução.

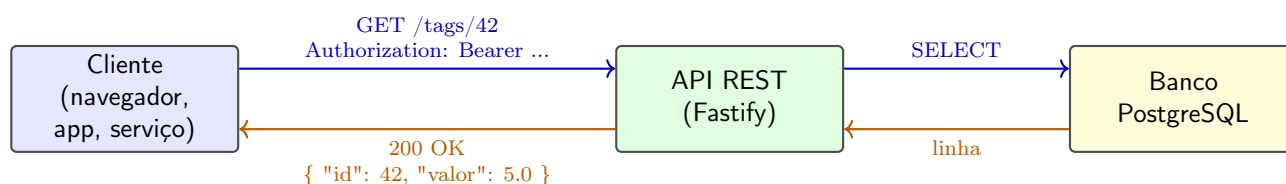
A combinação RDS + ECS + Lambda materializa uma arquitetura híbrida em que cada componente é usado em sua zona de melhor custo-benefício: o banco de dados permanece em um serviço estável e otimizado para persistência relacional; os microserviços de carga contínua executam em contêineres com previsibilidade de custo; e as tarefas esporádicas usam funções sob demanda, evitando o custo de manter servidores ociosos. Toda a infraestrutura é descrita como código em arquivos do AWS CloudFormation ou do Terraform, permitindo reprodução do ambiente em diferentes contas e regiões geográficas.

2.16 HTTP, API REST e JSON

O protocolo HTTP (*Hypertext Transfer Protocol*) é a base da comunicação na web e, por extensão, a base da comunicação entre os serviços de qualquer sistema distribuído moderno, incluindo o sistema supervisorio proposto. Trata-se de um protocolo cliente-servidor sem estado (*stateless*), em que cada requisição contém todas as informações necessárias para ser processada, sem dependência de requisições anteriores. Uma requisição HTTP é composta por método (GET, POST, PUT, PATCH, DELETE, entre outros), caminho do recurso (URL), cabeçalhos (*headers*) e, opcionalmente, corpo (*body*). A resposta contém código de status (200, 201, 400, 401, 404, 500 etc.), cabeçalhos e corpo.

Sobre o HTTP, o padrão **REST** (*Representational State Transfer*), proposto por Roy Fielding em sua tese de doutorado em 2000, estabelece um conjunto de restrições arquiteturais para projetar APIs web bem comportadas. Os princípios centrais incluem: identificação clara de recursos por URL (`/equipamentos/42/tags`); uso dos métodos HTTP de forma semântica (GET para consulta, POST para criação, PUT para atualização completa, PATCH para atualização parcial, DELETE para remoção); representação independente da forma interna (frequentemente JSON na comunicação cliente-servidor moderna); ausência de estado de sessão no servidor; e estrutura hierárquica de recursos que reflete o domínio do problema. APIs REST bem desenhadas são previsíveis, navegáveis e composíveis, propriedades alinhadas com o paradigma de orientação a serviços discutido por Bangemann *et al.* (2014) no contexto de automação industrial.

Figura 18: Ciclo requisição-resposta HTTP em uma API REST



Fonte: Autor.

O formato **JSON** (*JavaScript Object Notation*) é a representação de dados padrão de fato para APIs REST modernas. Trata-se de uma sintaxe textual derivada do JavaScript, simples de ler e escrever para humanos e fácil de gerar e parsear para máquinas. Suporta tipos primitivos (*string*, número, booleano, nulo) e estruturas compostas (objeto e *array*), o que se mostrou suficiente para a grande maioria dos cenários de troca de dados. JSON substituiu, em larga medida, o XML em comunicações web devido à sua menor verbosidade e ao acoplamento natural com o JavaScript dos navegadores. Adicionalmente, esquemas JSON (*JSON Schema*) permitem descrever formalmente o formato esperado de mensagens, oferecendo validação e geração de documentação automaticamente, como discutido na Subseção 2.14.

Em sistemas industriais conectados, a comunicação por API REST sobre HTTP é hoje uma alternativa cada vez mais comum ao acoplamento direto ao protocolo Modbus. Como observa Bangemann *et al.* (2014), a exposição de funcionalidades de dispositivos de campo como serviços web (*Web Services*) é um dos vetores que viabilizam o achatamento da pirâmide ISA-95 discutido na Subseção 2.5. No sistema proposto neste trabalho, os *gateways* de campo continuam usando Modbus para falar com os CLPs (por questões de compatibilidade e desempenho na rede industrial), mas todos os serviços de nuvem comunicam-se entre si e com o *frontend* via REST/JSON, em uma arquitetura tipicamente moderna.

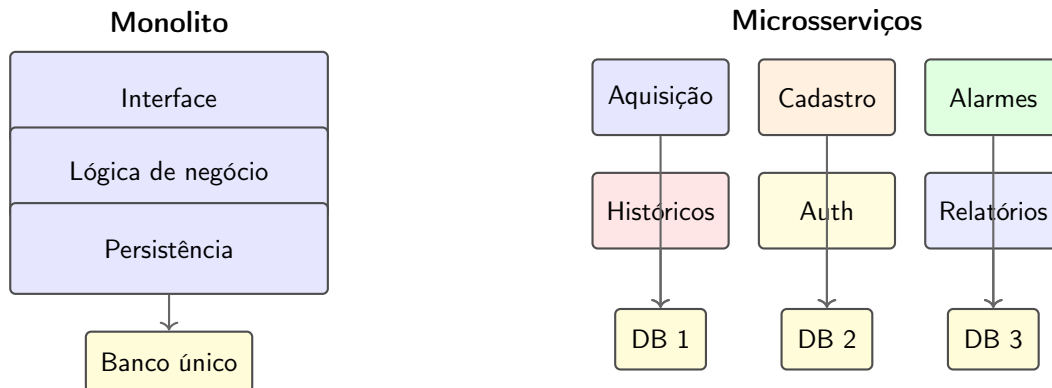
A escolha por REST sobre HTTP, em detrimento de alternativas como gRPC, GraphQL ou WebSockets puros, leva em conta a maturidade do ecossistema de bibliotecas, ferramentas e práticas, a facilidade de depuração (qualquer navegador ou cliente HTTP pode inspecionar requisições) e o alinhamento com o conhecimento prévio da equipe de desenvolvimento. WebSockets são usados em paralelo, exclusivamente para o canal de *streaming* de dados em tempo real entre o *backend* e as telas de supervisão ao vivo, em que a sobrecarga de uma nova requisição HTTP para cada atualização seria proibitiva.

2.17 Microserviços

O paradigma de **microserviços** é um estilo arquitetural em que uma aplicação é estruturada como um conjunto de serviços pequenos, autônomos e independentemente implantáveis, cada um responsável por um domínio bem delimitado e comunicando-se com os demais por meio de protocolos leves, tipicamente APIs HTTP/REST (Subseção 2.16) ou mensageria assíncrona. O termo popularizou-se a partir de 2014, embora suas ideias fundamentais sejam herdeiras diretas da *Service-Oriented Architecture* (SOA) discutida por Bangemann *et al.* (2014) no contexto industrial. A diferença essencial é uma questão de granularidade e autonomia: enquanto a SOA tradicional opera com serviços relativamente grandes orquestrados por barramentos centralizados (ESB), os microserviços apostam em serviços pequenos, com bancos próprios, ciclos de vida independentes e comunicação coreografada.

A motivação prática para arquiteturas de microsserviços, em contraposição a aplicações monolíticas tradicionais, vem das limitações que estas últimas apresentam à medida que crescem. Em um monolito, o código de todas as responsabilidades (autenticação, cadastro, aquisição, alarmes, históricos, interface) vive em um único processo, é desenvolvido por uma única equipe, é versionado em conjunto, é implantado de uma vez e escala como um todo. À medida que o sistema ganha funcionalidades, a base de código se torna difícil de evoluir sem efeitos colaterais, o tempo de implantação cresce, falhas em uma área podem comprometer o sistema todo, e a escalabilidade horizontal se aplica ao todo em vez de à parte realmente sobrecarregada.

Figura 19: Aplicação monolítica (esquerda) versus arquitetura de microsserviços (direita)



Fonte: Elaborado pelo autor com base em Bigheti (2020).

As vantagens centrais dos microsserviços são, segundo síntese da literatura técnica e da tese de Bigheti (2020), três. Primeiro, **implantação independente**: cada serviço pode ser atualizado, reiniciado ou substituído sem afetar os demais, o que reduz o risco de mudanças e encurta o ciclo de entrega. Segundo, **escalabilidade granular**: o serviço que se torna gargalo pode ser replicado isoladamente, sem necessidade de replicar toda a aplicação. Terceiro, **heterogeneidade tecnológica**: cada serviço pode adotar a linguagem, o *framework* e o banco mais adequados ao seu domínio, permitindo escolha técnica caso a caso em vez de imposição uniforme.

No contexto específico da Indústria 4.0, Bigheti (2020) defende em sua tese de doutorado que microsserviços são especialmente alinhados aos princípios da própria I4.0: a **modularidade** dos microsserviços corresponde à modularidade exigida das fábricas inteligentes; a **descentralização** das decisões em cada serviço espelha a descentralização defendida pela arquitetura plana baseada em serviços (Subseção 2.5); a **orientação a serviços** é princípio de projeto explícito da I4.0 segundo Pisching (2017); e a **interoperabilidade** via APIs padronizadas é exatamente o mecanismo pelo qual sistemas heterogêneos passam a se comunicar em uma cadeia de valor digital. A tese de Bigheti é, portanto, um dos pilares conceituais que sustenta a escolha arquitetural deste trabalho.

É preciso, no entanto, reconhecer as desvantagens e os custos do paradigma. A complexidade sai da aplicação e migra para o ecossistema operacional: torna-se necessário gerenciar dezenas de serviços, com seus contêineres (Subseção 2.12), suas réplicas, seus *deploys*, sua observabilidade, suas configurações e seus bancos de dados. A consistência de dados, antes garantida por transações ACID locais, passa a depender de mecanismos de consistência eventual e padrões como *Saga*. A latência da rede entra no caminho crítico de operações antes locais. Padrões como *API Gateway* (ponto único de entrada externo), *Service Registry* (descoberta automática

de serviços), *Circuit Breaker* (proteção contra cascatas de falha) e *Distributed Tracing* (rastreamento de requisições que atravessam vários serviços) tornam-se essenciais. Bigheti (2020) dedica capítulos próprios à discussão desses padrões aplicados a sistemas industriais.

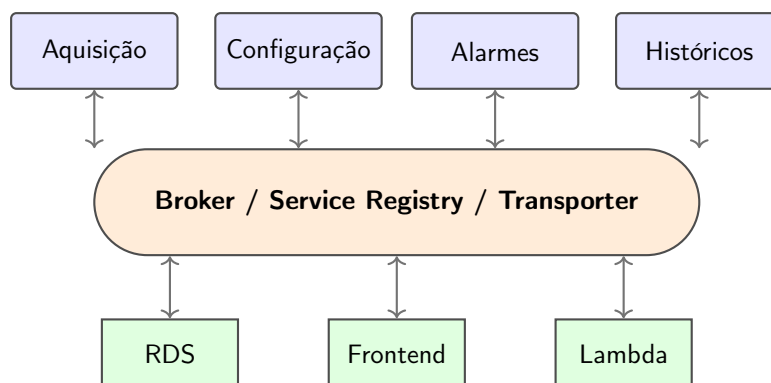
No sistema supervisorio aqui proposto, a decomposição em microsserviços foi guiada pelo princípio de *bounded contexts* do *Domain-Driven Design*: cada serviço tem responsabilidade clara sobre um domínio do problema, identificada pelos casos de uso e pelas entidades centrais (Subseção 2.10). O serviço de aquisição é o orquestrador dos ciclos Modbus; o de configuração mantém o cadastro; o de alarmes consome eventos de aquisição e dispara notificações; o de históricos persiste leituras para consulta posterior; o de autenticação cuida de identidade e direitos. A comunicação entre eles é orquestrada pelo *framework* Moleculer, detalhado na Subseção 2.18, que oferece descoberta, balanceamento e tolerância a falhas em uma única biblioteca leve.

2.18 Moleculer

O Moleculer é um *framework* de microsserviços para Node.js, projetado para construir sistemas distribuídos compostos por serviços pequenos, focados e independentemente implantáveis. Posiciona-se como uma alternativa pragmática a soluções mais pesadas (Spring Cloud no ecossistema Java, por exemplo), oferecendo descoberta de serviços, balanceamento de carga, tolerância a falhas, transações distribuídas e diversos transportes (TCP, NATS, Redis, MQTT, Kafka) em uma única biblioteca leve.

A motivação para uma arquitetura de microsserviços neste trabalho deriva diretamente da literatura sobre Indústria 4.0. Bigheti (2020), em sua tese de doutorado sobre arquitetura de automação e controle orientada a microsserviços para a Indústria 4.0, defende que a granularidade fina dos microsserviços, sua autonomia operacional e sua capacidade de evolução independente são alinhadas com os princípios de modularidade, descentralização e orientação a serviços da própria I4.0 (Subseção 2.8). Em vez de um monolito que centraliza aquisição, supervisão, persistência, alarmes e relatórios em um único processo, o sistema é decomposto em serviços coesos que se comunicam por mensagens.

Figura 20: Arquitetura de microsserviços baseada em barramento de eventos (broker Moleculer)



Fonte: Elaborado pelo autor com base em Bigheti (2020).

No modelo do Moleculer, cada **serviço** é uma classe (ou objeto) que define um conjunto de **actions** (ações invocáveis remotamente, similares a métodos de uma API) e **events** (mensagens publicadas para outros serviços ouvirem). Os serviços se registram em um **broker** que

faz a descoberta automática dos nós da rede e roteia chamadas e eventos para a instância adequada, com balanceamento de carga entre múltiplas instâncias do mesmo serviço quando há replicação. Quando um serviço falha, o broker pode redirecionar a chamada para outra instância, implementar *circuit breakers* para evitar cascatas de falha, ou retornar respostas *fallback* previamente cadastradas.

O Moleculer suporta múltiplos transportes para a comunicação entre nós. Em ambiente de desenvolvimento local, frequentemente usa-se TCP direto. Em produção, recomenda-se barramentos resilientes como NATS, Redis Pub/Sub ou MQTT, que oferecem persistência opcional de mensagens, entrega garantida e tolerância a falhas no próprio barramento. O sistema supervisorio proposto usa NATS pela combinação de latência baixa, simplicidade operacional e suporte nativo a streams persistentes, valiosos para reprocessar eventos de alarme ou de aquisição quando um serviço se recupera de uma falha.

A divisão do sistema em microserviços feita neste trabalho segue o princípio de *bounded contexts* do *Domain-Driven Design*: cada serviço tem responsabilidade clara sobre um domínio do problema. O serviço de **aquisição** é o orquestrador dos ciclos Modbus, traduz leituras brutas para unidades de engenharia (Subseção 2.3) e publica eventos com os valores adquiridos. O serviço de **configuração** mantém o cadastro de equipamentos, *tags* e parâmetros. O serviço de **alarmes** consome os eventos de aquisição, compara com limites configurados e emite eventos de alarme quando aplicável. O serviço de **históricos** consome todos os eventos relevantes e persiste no banco para consulta posterior. O serviço de **autenticação** cuida da identidade e dos direitos de acesso de usuários e sistemas integrados.

2.19 Software como Serviço (SaaS)

O Software como Serviço, ou *Software as a Service* (SaaS), é a camada mais alta da pirâmide de serviços em nuvem descrita na Subseção 2.9. Diferentemente de IaaS, que entrega *hardware* virtualizado, e de PaaS, que entrega plataformas de desenvolvimento, o SaaS entrega ao usuário final aplicações completas, executando integralmente na nuvem do provedor e acessadas por meio de portais web ou clientes leves (Henriques da Silva, 2010). O usuário não precisa instalar nem manter *software* localmente; basta um terminal conectado à internet, conforme a metáfora do “terminal burro” originalmente associada à computação nas nuvens (Henriques da Silva, 2010).

O modelo de cobrança típico do SaaS é por assinatura, em ciclos mensais ou anuais, frequentemente escalonado por número de usuários, número de funcionalidades habilitadas ou volume de dados processados, o que se alinha à característica de *serviço medido* definida nas características essenciais da computação em nuvem (Pedrosa; Nogueira, 2011). Como o *software* é instalado uma única vez na infraestrutura do provedor e atende a múltiplos clientes simultaneamente em arquitetura *multi-tenant*, o ganho de escala permite que o custo unitário por cliente seja substancialmente inferior ao da venda tradicional de licenças *on-premises*, o que viabiliza adoção por organizações de menor porte que antes não tinham acesso a esse tipo de tecnologia.

Exemplos consagrados de SaaS incluem suítes de produtividade (Google Workspace, Microsoft 365), gestão de relacionamento com clientes (Salesforce), comunicação corporativa (Slack, Microsoft Teams), edição de imagens online (Photoshop on the web) e armazenamento e sincronização de arquivos (Dropbox, Google Drive) (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011). No segmento industrial, soluções de supervisão remota, manutenção preditiva, telemetria de frota e gestão de manufatura têm migrado progressivamente para o modelo SaaS, aproveitando as vantagens de elasticidade, manutenção centralizada e atualização contínua, conforme defendido por Lima (2026) no contexto da transformação digital industrial.

A arquitetura *multi-tenant* é o mecanismo técnico que viabiliza o modelo SaaS economicamente. Em uma instalação tradicional, cada cliente teria uma cópia separada do *software* executando em sua infraestrutura. No SaaS, uma única instância da aplicação atende a vários clientes simultaneamente, com isolamento lógico dos dados por meio de identificadores de *tenant* em todas as consultas ao banco, criptografia separada por cliente e políticas de controle de acesso que impedem qualquer vazamento entre inquilinos. Esse desenho exige cuidado redobrado com segurança, mas oferece vantagens operacionais expressivas: atualizações são aplicadas uma única vez para todos os clientes, métricas e *logs* são unificados, e a infraestrutura é dimensionada de forma agregada.

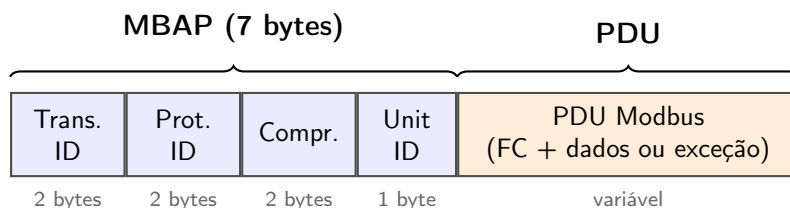
O sistema supervisorio proposto neste trabalho é, por desenho, uma solução SaaS para o setor de saneamento e processos industriais correlatos. Cada cliente contratante (uma concessionária de água, por exemplo) é um *tenant* dentro da mesma infraestrutura, com seus equipamentos, *tags*, históricos, usuários e configurações isolados logicamente dos demais. A operação cotidiana, a manutenção evolutiva, as atualizações de funcionalidades, as correções de segurança e o monitoramento da infraestrutura são responsabilidades centralizadas do fornecedor, e o cliente acessa o sistema exclusivamente por meio do navegador web e de credenciais individuais. Esse posicionamento alinha-se diretamente com a tendência mais ampla descrita por Bigheti (2020) de migração da automação industrial para modelos orientados a serviços, e materializa, na prática, a visão de fábrica inteligente conectada à nuvem associada à Indústria 4.0 (Pisching, 2017).

2.20 Protocolo Modbus TCP e Modbus RTU

O Modbus surgiu como protocolo de requisição/resposta entre um cliente (*master*) e um ou mais servidores (*slaves*), hoje também denominados dispositivos de campo. Na supervisão, o sistema na nuvem atua como cliente: envia requisições que leem ou escrevem registradores e *coils* no escravo; o escravo responde com dados ou com código de exceção quando a operação não é permitida. As funções mais empregadas em telemetria incluem leitura de *holding registers* (Função 03), leitura de *input registers* (Função 04), leitura de *coils* (Função 01), leitura de entradas discretas (Função 02) e escrita em *holding registers* ou *coils* (Funções 06, 16, 05, 15, entre outras), conforme o fabricante do CLP ou medidor exponha o mapa de memória.

O **Modbus TCP** (especificado na guia de implementação da Modbus Organization) transporta a PDU Modbus (*Protocol Data Unit*) dentro de uma conexão TCP/IP, precedida pelo cabeçalho MBAP (*Modbus Application Protocol*). O MBAP contém identificador de transação (para emparelhar requisição e resposta quando há várias pendentes), identificador de protocolo (valor zero para Modbus), campo de comprimento (número de bytes seguintes, incluindo o *Unit ID* e a PDU) e o *Unit ID*, que seleciona o escravo quando há multiplexer ou ponte serial/TCP no caminho. A Figura 21 resume essa composição; a integridade no enlace TCP é delegada à pilha IP (checksums de camada de transporte e rede), não havendo CRC Modbus no quadro TCP.

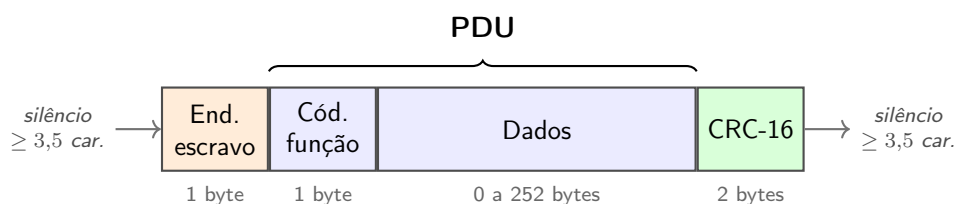
Figura 21: Composição lógica do Modbus TCP: cabeçalho MBAP seguido da PDU (código de função, dados ou exceção)



Fonte: Elaborado pelo autor com base em Modbus Organization (2012a).

O **Modbus RTU** transmite a mesma PDU (sem o MBAP), em geral sobre RS-485 multi-ponto, com enquadramento binário e delimitação de mensagens por tempo de silêncio na linha (*inter-frame*). Cada quadro contém endereço do escravo (1 byte), código de função, dados e dois bytes de CRC-16 sobre todo o trecho precedente ao CRC; o receptor recalcula o CRC para detectar ruídos ou colisões na serial. A Figura 22 ilustra essa estrutura. Em telemetria com modem ou conversor, é frequente o trecho WAN ser IP enquanto o CLP continua em RTU: o *gateway* ou o modem encapsula o quadro RTU dentro de TCP (túnel transparente) ou converte TCP para RTU, mantendo o *Unit ID* coerente com o endereço serial do escravo.

Figura 22: Estrutura simplificada do quadro Modbus RTU (delimitação temporal entre quadros)



Fonte: Elaborado pelo autor com base em Modbus Organization (2012b).

Em resumo, TCP e RTU diferem principalmente pelo meio e pelo enquadramento: no TCP, endereçamento e multiplexação usam conexão IP, porta e MBAP; no RTU, usam-se endereço de escravo na serial, tempos entre quadros e CRC explícito. Na arquitetura deste trabalho, o serviço na nuvem negocia TCP com o *gateway*; a partir daí, conforme o cadastro do ponto, a mesma semântica Modbus pode chegar ao equipamento como TCP nativo (por exemplo, CLP com Ethernet e servidor Modbus TCP) ou como RTU no barramento local, caso típico de instrumentos e CLPs com porta serial isolada ao modem.

2.21 Modelagem, serviço de leitura e agendamento

O cadastro no banco de dados estrutura a aquisição em níveis hierárquicos (por exemplo, unidade operacional, equipamento, conjunto de variáveis (*tags*) e parâmetros de comunicação) e armazena, para cada ponto, o mapeamento Modbus necessário: endereço da unidade (*Unit ID*), código de função (leitura de *holding registers*, *input registers*, *coils* ou *discrete inputs*), endereço inicial, quantidade de elementos, tipo de dado bruto (inteiro de 16 ou 32 bits, *float*, palavras múltiplas), ordem de bytes/*words* quando relevante, fatores de escala e *offset* para

conversão em unidade física, limites de engenharia e classificação (leitura periódica, leitura sob demanda, escrita supervisionada). Essa modelagem permite que o mesmo serviço de aquisição atenda processos distintos apenas por configuração, sem alteração de código para cada novo medidor ou CLP.

O serviço de aquisição implementa o ciclo “ler configuração → compor telegramas Modbus → enviar → validar resposta → decodificar → publicar resultado”. Na etapa de validação, verifica-se coerência de tamanho, exceções Modbus (códigos de erro de função), tempos de resposta e integridade básica dos dados; na decodificação, aplicam-se máscaras, combinações de registradores e regras de engenharia para obter grandezas como vazão instantânea, nível filtrado ou estado discreto de um conjunto de bits. Quando a leitura falha (timeout, perda de enlace, recusa do escravo), o serviço pode registrar a falha, manter o último valor válido com indicador de “desatualizado” e alimentar mecanismos de alarme de comunicação na supervisão. Esse cuidado com a indisponibilidade do dado é aspecto central em SCADA, onde a ausência de dado precisa ser tão visível quanto um limite de processo ultrapassado.

Em redes móveis e enlaces compartilhados, políticas conservadoras de concorrência evitam disparar centenas de requisições simultâneas contra o mesmo *gateway* ou o mesmo modem. Uma abordagem plausível é limitar o número de sessões TCP paralelas, agrupar leituras em blocos contíguos de registradores quando o equipamento permitir (reduzindo idas e vindas) e respeitar tempos mínimos entre telegramas exigidos por alguns conversores. Parâmetros como tempo limite (*timeout*), número de tentativas e *backoff* exponencial entre retentativas podem ser configurados por equipamento ou por site, equilibrando resiliência e carga na rede.

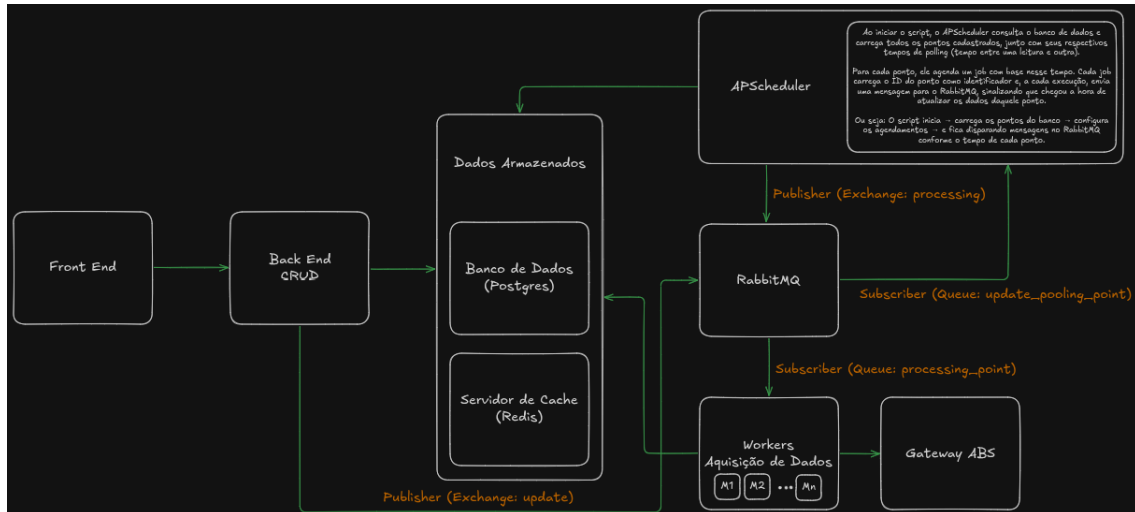
O agendamento da aquisição separa a decisão “quando ler” da execução “como ler”. Em sistemas de supervisão, nem todas as variáveis exigem a mesma cadência: grandezas lentas (nível em reservatório volumoso) podem ser amostradas em minutos, enquanto indicadores críticos (pressão de sucção de bomba, intertravamentos) podem exigir segundos. O agendador mantém filas ou *timelines* por periodicidade, evita sobreposição de ciclos longos para o mesmo equipamento quando a leitura anterior ainda não terminou e permite janelas de manutenção (redução temporária de frequência) ou leituras prioritárias (por exemplo, após comando de partida). Essa camada também pode coordenar leituras complementares: telemetria periódica para tendência e leituras sob demanda disparadas pela interface quando o operador abre uma tela detalhada.

Por fim, os valores adquiridos são associados a carimbo de tempo do servidor (instante em que a resposta foi recebida) e, quando disponível no equipamento, a referências temporais adicionais; em seguida seguem para persistência histórica e para os mecanismos de atualização das telas gráficas. Assim, a aquisição fecha o laço entre o processo físico e a representação digital monitorada pelo operador, mantendo rastreabilidade e condições para auditoria operacional.

3 Funcionamento

O funcionamento do sistema organiza-se em camadas, conforme ilustrado na Figura 23. Essa organização separa responsabilidades entre a interface de supervisão, os serviços de domínio executados na nuvem, a persistência das informações e a conectividade com os equipamentos instalados.

Figura 23: Arquitetura do Sistema



Fonte: Autor.

A Figura 23 detalha a arquitetura completa do sistema. O **frontend** é desenvolvido em Next.js (Subseção 2.13) e oferece a interface web de supervisão acessada pelos operadores em qualquer ponto da rede. O **backend CRUD** é construído em Fastify (Subseção 2.14) e expõe os **endpoints** REST para cadastro de equipamentos, configuração de *tags*, consulta de históricos e disparo de comandos. A camada de **dados armazenados** integra o banco relacional PostgreSQL, gerenciado pelo Amazon RDS (Subseções 2.10 e 2.15), com o servidor de cache Redis no Amazon ElastiCache (Subseção 2.11), reunindo persistência histórica de longa duração e acesso de baixa latência ao estado corrente de cada *tag*. O **APScheduler** é um servidor dedicado de agendamento que carrega no início da execução todos os pontos cadastrados no banco com seus respectivos tempos de *polling* e, para cada ponto, agenda um *job* próprio; a cada disparo, publica uma mensagem com o identificador do ponto, sinalizando que chegou a hora de atualizar aquele dado. O barramento entre os serviços é orquestrado pelo **Moleculer** (Subseção 2.18), que recebe os comandos de aquisição no canal **processing** e os encaminha aos **workers de aquisição** (M1, M2, ..., Mn). Esses workers consomem o canal **processing_point**, estabelecem sessão TCP com o *Gateway ABS*, executam a leitura Modbus correspondente conforme tratado nas Subseções 2.20 e 2.6, e devolvem o resultado ao barramento; o Moleculer então publica no canal **update** a leitura processada para que o banco persista o histórico e o cache atualize o valor corrente.

3.1 Validações dos Requisitos do SCADA

As subseções a seguir validam, item por item, o atendimento do sistema aos requisitos clássicos do SCADA descritos por Zanghi (2019) e revisados na Subseção 2.2. Cada validação mostra como a arquitetura ilustrada na Figura 23 e descrita acima atende aos seis pilares

definidos: aquisição e registro de dados, sincronismo de tempo, níveis de operação, comando e controle, escalabilidade e redundância.

3.1.1 Aquisição e Registro de Dados

No quesito de aquisição e registro de dados discutido na Subseção 2.2, o sistema atende a todos os requisitos descritos por Zanghi (2019). A aquisição ocorre por meio dos workers, que se comunicam com IEDs e UTRs via Modbus pelo *gateway* de borda e disponibilizam os valores em formato digital ao supervisório. Os pontos são cadastrados como *tags* no banco de dados, classificadas como digitais (entrada DI, saída DO, memória DM) para representar estados binários como liga/desliga de motores e abertura/fechamento de válvulas, ou analógicas (entrada AI, saída AO, memória AM) para grandezas multibit como medições de pressão, vazão, nível e corrente. O registro contínuo das leituras na base de dados PostgreSQL sustenta as análises posteriores em gráficos de tendência e em relatórios sequenciais de eventos digitais, enquanto o cache Redis mantém os valores correntes acessíveis para a renderização imediata das telas de supervisão sem onerar o banco principal a cada atualização.

3.1.2 Sincronismo de Tempo

O sincronismo de tempo exigido pelo SCADA é garantido pela combinação entre o relógio do APScheduler, sincronizado por NTP com a infraestrutura da AWS, e o carimbo de tempo aplicado pelos workers no instante exato em que a resposta Modbus é recebida do dispositivo de campo. Cada mensagem que trafega pelo barramento Moleculer carrega esse carimbo de tempo, que se preserva inalterado ao longo de toda a cadeia: do worker para o canal `update`, do canal para o banco e para o cache, do banco para o backend e do backend para o frontend. Assim, alarmes e eventos digitais captados em diferentes pontos do sistema recebem estampas de tempo coerentes na escala de milissegundos, atendendo à exigência de cronologia precisa para análises *a posteriori* de ocorrências e faltas, sobretudo em processos com dinâmica rápida.

3.1.3 Níveis de Operação

Os níveis de operação clássicos do SCADA mapeiam-se naturalmente sobre a arquitetura. O Nível 0, dos equipamentos de campo e seus sensores, corresponde aos transmissores, medidores e demais dispositivos físicos instalados nas unidades operacionais dos clientes. O Nível 1, da supervisão local junto aos IEDs, é representado pelos CLPs e modems no campo, que executam a comunicação Modbus com os instrumentos e, quando há, oferecem uma IHM simples para operação no próprio ponto. O Nível 2, da supervisão da instalação como um todo, é materializado pelos workers de aquisição, pelo Moleculer e pelos serviços de domínio na nuvem (cadastro, alarmes, históricos), que agregam os pontos relevantes de cada cliente em estações redundantes com telas elaboradas para supervisão completa da unidade. O Nível 3, do centro de operação remota, é o frontend Next.js acessado via navegador, que consolida pontos provenientes de múltiplas instalações geograficamente distribuídas, disponibilizando apenas o que é estritamente necessário para a gestão sistêmica.

3.1.4 Comando e Controle

O comando e controle previstos pelo SCADA são implementados nas três variantes descritas por Zanghi (2019). O telecomando instantâneo, em que o operador no frontend dispara mudança imediata do estado de um equipamento (acionamento de bomba, abertura de válvula,

comando a disjuntor), é traduzido em uma escrita Modbus pelo worker correspondente. O telecontrole, em que um *set point* é definido pelo operador para regular no tempo a resposta de um processo, percorre o mesmo caminho mas atualiza um *holding register* específico. O telecomando ou telecontrole programado, em que ações são agendadas conforme cenários previamente definidos, é mantido como configuração no banco e disparado pelo APScheduler no horário marcado, com o instante de execução registrado. As quatro fases do processamento (seleção, verificação, confirmação e execução) são distribuídas entre frontend (seleção e confirmação explícita por modal), backend Fastify (verificação de intertravamentos e condições de segurança) e worker (execução da escrita Modbus); após a execução, o próprio worker realiza uma leitura de verificação para confirmar que a solicitação foi aceita pelo dispositivo de campo, e, em caso de falha, gera o alarme correspondente para sinalizar a inconsistência ao operador.

3.1.5 Escalabilidade

A escalabilidade é atendida em três frentes complementares. Na expansão de cadastro, novos pontos e novas telas podem ser adicionados em produção sem alteração de código: o worker é parametrizado pelo banco (Subseção 2.10) e o frontend renderiza as telas dinamicamente conforme a configuração; o APScheduler reage à inclusão recarregando a lista de pontos e ajustando seus *jobs*. Na capacidade de processamento, os workers (M1, M2, ..., Mn) podem ser replicados horizontalmente para distribuir a carga: o Moleculer faz balanceamento automático entre as instâncias que consomem o canal `processing_point`, e o Amazon ECS (Subseção 2.15) provisiona novas réplicas dos contêineres Docker conforme métricas de utilização. Na infraestrutura de dados, os serviços gerenciados Amazon RDS e ElastiCache escalam verticalmente sob demanda. Esse desenho elimina o gargalo de licenciamento por número de pontos típico de supervisórios comerciais e permite acompanhar a expansão das instalações sem reorçamento da solução, ainda que cuidados de capacidade dos elementos de rede e do barramento Moleculer devam ser considerados desde o projeto.

3.1.6 Redundância

A redundância acompanha a arquitetura em todas as camadas críticas, contemplando as recomendações de Zanghi (2019) e Salvador (2019a). Os workers de aquisição operam como uma frota em que cada instância consome trabalho de uma mesma fila gerenciada pelo Moleculer, de modo que a perda de um worker é absorvida pelos demais sem perda de leituras pendentes. O banco de dados em RDS é configurado com réplica em zona de disponibilidade distinta, com *failover* automático em caso de falha do nó primário e *snapshots* programados para recuperação ponto-a-ponto (Subseção 2.10). O cache Redis opera no modo *cluster* com réplicas espelhadas (Subseção 2.11). O frontend Next.js e o backend Fastify são executados em múltiplos contêineres atrás de um balanceador de carga, e o próprio barramento Moleculer pode ser configurado em *cluster* para tolerância a falhas no transporte. Esse conjunto de medidas atinge níveis de confiabilidade compatíveis com a criticidade exigida em aplicações industriais sensíveis.

3.2 Servidores

O sistema decompõe-se em sete componentes lógicos de servidor, cada qual com responsabilidade bem delimitada e ciclo de vida independente, em alinhamento com o paradigma de microsserviços discutido na Subseção 2.17 e defendido por Bigheti (2020) para sistemas industriais orientados a serviços. Todos os componentes são empacotados como contêineres Docker (Subseção 2.12) e orquestrados pelo Amazon ECS (Subseção 2.15), o que permite implantação

independente, escalabilidade granular e isolamento de dependências. Cada um dos sete servidores é descrito nas subseções a seguir quanto à tecnologia escolhida, à sua responsabilidade no sistema e às interfaces que mantém com os demais componentes.

3.2.1 Frontend

O servidor de frontend é desenvolvido em Next.js (Subseção 2.13) e materializa a interface de supervisão acessada pelo operador via navegador. Suas telas incluem visualização de pontos ao vivo, dashboards de tendência histórica, cadastro de equipamentos e *tags*, painéis de alarmes ativos e formulários administrativos. As telas de operação em tempo real adotam renderização no cliente com atualização incremental via WebSocket, garantindo latência baixa entre a leitura no campo e a apresentação ao operador; as telas de cadastro e relatórios adotam renderização no servidor (SSR), aproveitando o pré-processamento e a verificação de credenciais. A comunicação com o restante do sistema ocorre exclusivamente via APIs REST/JSON (Subseção 2.16) e WebSocket, sem acoplamento direto com o banco ou com o barramento de eventos.

3.2.2 Backend

O servidor de backend CRUD é construído sobre o *framework* Fastify (Subseção 2.14) e expõe os *endpoints* REST consumidos pelo frontend. Suas atribuições incluem operações de criação, leitura, atualização e remoção sobre o cadastro de clientes, unidades, equipamentos e *tags*; consultas históricas com filtros de tempo, equipamento e ponto; gestão de usuários, perfis e permissões; e disparo de comandos de telecontrole pelo operador. Cada requisição é validada contra um esquema JSON Schema declarativo, autenticada por *token* JWT armazenado no Redis e auditada em *log* estruturado. Quando uma operação envolve interação com o campo (por exemplo, escrita Modbus), o backend não fala diretamente com os workers, mas publica um comando no barramento Moleculer, mantendo o desacoplamento entre a camada de aplicação e a camada de aquisição.

3.2.3 Banco de Dados

O servidor de banco de dados é uma instância PostgreSQL gerenciada pelo Amazon RDS (Subseções 2.10 e 2.15), responsável pela persistência de longa duração de todo o estado relevante do sistema: cadastro hierárquico (cliente, unidade operacional, equipamento, *tag*), histórico de leituras com carimbo de tempo, eventos de alarme, registros de auditoria, configuração de usuários e parâmetros de comunicação. O esquema relacional aproveita chaves estrangeiras para garantir integridade referencial; tabelas históricas são particionadas por intervalo de tempo para evitar tabelas monolíticas, e índices nos campos *tag_id* e *timestamp* aceleram as consultas mais frequentes. A configuração em produção emprega réplica em zona de disponibilidade distinta, com *failover* automático em caso de falha do nó primário e *snapshots* programados para recuperação ponto-a-ponto, conforme recomendado por Salvador (2019a).

3.2.4 Cache

O servidor de cache é uma instância Redis provida pelo Amazon ElastiCache (Subseção 2.11) e cumpre três papéis simultâneos. Como cache de leituras frequentes, mantém em memória o valor corrente de cada *tag* ativa, acessado pelas telas de supervisão em tempo real sem onerar o banco principal a cada atualização. Como barramento publicador-assinante de baixa latência, distribui eventos de aquisição aos consumidores interessados (alarmes, históricos, frontend ao

vivo). Como armazenamento efêmero, abriga *tokens* de autenticação, contadores de limitação de requisições por usuário (*rate limiting*) e travas distribuídas que evitam processamento duplicado de comandos críticos. O cluster opera com réplicas espelhadas em zonas distintas e expiração automática (TTL) das chaves voláteis.

3.2.5 APScheduler

O servidor de agendamento executa a biblioteca APScheduler em um contêiner dedicado e é responsável por decidir o instante em que cada ponto deve ser lido. No início da execução, o serviço consulta o banco de dados, carrega todos os pontos cadastrados com seus respectivos tempos de *polling* (intervalo entre uma leitura e a próxima) e agenda um *job* próprio para cada ponto com base nesse tempo. A cada disparo, o *job* carrega o identificador do ponto e publica uma mensagem no barramento Moleculer, sinalizando aos workers que chegou a hora de atualizar aquele dado. O servidor reage a mudanças no cadastro (inclusão, remoção ou alteração de periodicidade) por meio de mecanismo de invalidação que recarrega os *jobs* afetados, sem necessidade de reinício; janelas de manutenção e leituras prioritárias também são tratadas nessa camada, separando a decisão “quando ler” da execução “como ler”.

3.2.6 Barramento Moleculer

O servidor de barramento de mensagens é uma instância do Moleculer (Subseção 2.18) que orquestra a comunicação assíncrona entre os demais componentes. Três canais principais estruturam o tráfego: o canal **processing** recebe comandos de aquisição emitidos pelo APScheduler e pelo backend (no caso de leituras sob demanda); o canal **processing_point** é consumido pelos workers de aquisição em modo de balanceamento de carga; e o canal **update** carrega os resultados das leituras de volta para o banco e para o cache, além de servir como gatilho para os serviços de alarmes e de notificação. O Moleculer também fornece descoberta automática de serviços, balanceamento entre instâncias replicadas, *circuit breakers* para isolamento de falhas em cascata e *distributed tracing* para rastreabilidade fim a fim das requisições que atravessam vários serviços.

3.2.7 Workers de Aquisição

Os workers de aquisição (M1, M2, ..., Mn) constituem a frota responsável pela comunicação efetiva com o campo. Cada worker é uma instância idêntica de um contêiner Docker, replicável horizontalmente conforme a carga, e mantém um conjunto de sessões TCP ativas com os *gateways* ABS de cada cliente. Ao consumir uma mensagem do canal **processing_point**, o worker monta o telegrama Modbus correspondente (com base nos parâmetros do ponto recuperados do cadastro), envia a requisição pelo *gateway*, valida a resposta, aplica a conversão para unidade de engenharia descrita na Subseção 2.3 e publica o resultado no canal **update**. Os workers são *stateless*, o que viabiliza adição ou remoção de instâncias sem perda de estado; políticas de *timeout*, retentativa e *backoff* exponencial são parametrizadas por equipamento, mantendo a robustez frente a falhas transitórias de comunicação.

3.3 Aquisição de Dados

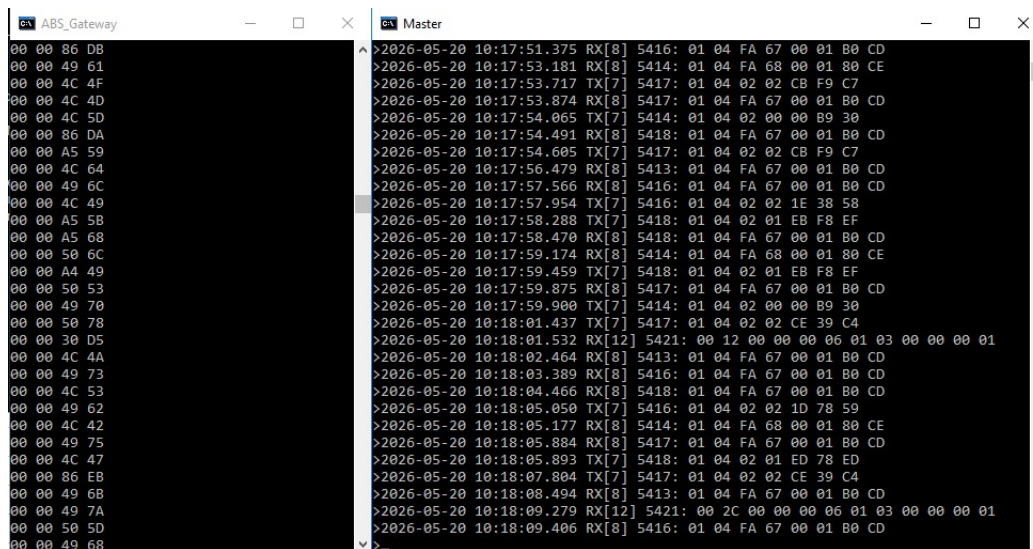
A camada de aquisição de dados é o subsistema responsável por traduzir o estado físico do processo (níveis, vazões, pressões, energia, status de bombas e válvulas, alarmes, contadores etc.) em valores digitais confiáveis, com referência temporal e com metadados suficientes para

consumo pela supervisão, por relatórios operacionais e por rotinas de histórico. Em termos práticos, ela engloba desde a montagem das requisições até a entrega dos dados já convertidos para unidades de engenharia (valores nas unidades físicas usuais na operação, como metro, m³/h, bar ou kW). Com frequência, o equipamento expõe a grandeza apenas como inteiro em um ou mais registradores Modbus; o programa do CLP grava então uma representação em ponto fixo (por exemplo, o nível em centímetros com uma casa decimal vem como inteiro que deve ser dividido por 10, ou a vazão usa divisor 100), e cabe à camada de aquisição aplicar a multiplicação ou divisão, somas de deslocamento (*offset*) e demais fatores de escala cadastrados para reproduzir o valor real usado na operação, prontos para gravação em banco.

Por operar em ambiente de nuvem, a arquitetura prevê um ponto de concentração na borda da solução (o *gateway*) e dispositivos de campo (os modems), que mantêm a sessão com a infraestrutura central e encaminham o tráfego até o equipamento industrial conectado localmente, via serial RS-485 ou Ethernet. O *gateway* recebe múltiplas conexões TCP vindas dos serviços na nuvem e, com base em regras de roteamento, direciona cada fluxo ao modem associado a um ponto remoto. O modem, por sua vez, materializa o enlace de longa distância (GPRS ou outro meio de acesso à internet) e encerra a parte WAN da comunicação, entregando ao equipamento final a mensagem no formato esperado.

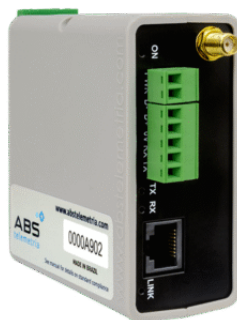
Na prática, é comum encontrar soluções integradas por fornecedores de telemetria, a título de exemplificação, o *gateway* ABS (Figura 24) e o modem GPRS ABS Cel X (Figura 25) representam a camada física de conectividade adotada em muitos projetos: o primeiro agrega e roteia sessões, o segundo viabiliza o acesso móvel em um ponto de medição remoto.

Figura 24: Gateway ABS



Fonte: Autor.

Figura 25: Modem GPRS ABS Cel X



Fonte: Autor.

Entre a nuvem e o equipamento de campo, o protocolo de aplicação adotado para leitura e escrita das variáveis de processo é o Modbus. O encadeamento completo (serviço de aquisição, *gateway*, modem e dispositivo escravo) preserva a semântica das funções Modbus (códigos de função, endereços de registradores ou *coils*, dados de resposta), variando apenas o “envelope” de transporte: quadros TCP/IP na WAN e, no trecho serial, quadro RTU com verificação por CRC. O modo TCP e o modo RTU, bem como o encapsulamento usual em arquiteturas com telemetria, são tratados na Subseção 2.20.

4 Cronograma

O cronograma de atividades previstas para o desenvolvimento deste Trabalho de Conclusão de Curso é apresentado na Tabela 2, distribuindo as atividades ao longo dos doze meses do ano, organizadas conforme as fases de Projeto de Trabalho de Conclusão, Trabalho de Conclusão e Defesa.

Tabela 2: Cronograma de atividades do TCC.

ATIVIDADE	MÊS											
	01	02	03	04	05	06	07	08	09	10	11	12
Definição do Tema	X											
Pesquisa Bibliográfica		X	X	X				X	X	X		
Elaboração do Projeto de Trabalho de Conclusão			X	X	X	X						
Entregas ao Orientador				X	X	X		X	X	X	X	
Apresentação do Projeto de Trabalho de Conclusão						X						
Elaboração do Trabalho de Conclusão							X	X	X	X	X	X
Apresentação do Trabalho de Conclusão												X
Defesa do Trabalho de Conclusão												X

Fonte: Autor.

Referências

- ABBAS, Hosny Ahmed. **Efficient Web-Based SCADA System**. 2011. Dissertação (Mestrado em Engenharia Elétrica) – Faculty of Engineering, South Valley University, Aswan.
- BANGEMANN, Thomas *et al.* State of the Art in Industrial Automation. *In*: COLOMBO, Armando W. *et al.* (ed.). **Industrial Cloud-Based Cyber-Physical Systems: The IMC-AESOP Approach**. Cham: Springer, 2014. p. 23–47. ISBN 978-3-319-05623-4. DOI: 10.1007/978-3-319-05624-1_2.
- BIGHETI, Jeferson André. **Arquitetura de Automação e Controle Orientada a Microserviços para a Indústria 4.0**. 2020. Tese (Doutorado em Engenharia Elétrica) – Faculdade de Engenharia, Universidade Estadual Paulista “Júlio de Mesquita Filho”, Bauru.
- DAMIN, Délio. **Melhores práticas para o desenvolvimento de uma aplicação Elipse E3**. Atualizado em 22 dez. 2022. Elipse Software. 7 jul. 2022. Disponível em: <https://kb.elipse.com.br/melhores-praticas-para-o-desenvolvimento-de-uma-aplicacao-elipse-e3/>. Acesso em: 20 maio 2026.
- GAUSHELL, Dennis J.; DARLINGTON, Henry T. Supervisory control and data acquisition. **Proceedings of the IEEE**, v. 75, n. 12, p. 1645–1658, 1987. DOI: 10.1109/PROC.1987.13932.
- HENRIQUES DA SILVA, Fabrício Rodrigues. **Um estudo sobre os benefícios e os riscos de segurança na utilização de Cloud Computing**. 2010. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Centro Universitário Augusto Motta, Rio de Janeiro.
- LASI, Heiner *et al.* Industry 4.0. **Business & Information Systems Engineering**, v. 6, n. 4, p. 239–242, 2014. DOI: 10.1007/s12599-014-0334-4.
- LIMA, Samir. **A Ponte entre a Intuição Humana e a Autonomia Industrial: O Papel Estratégico do EPM Processor na Transformação Digital**. Elipse Software. 9 fev. 2026. Disponível em: <https://kb.elipse.com.br/a-ponte-entre-a-intuicao-humana-e-a-autonomia-industrial-o-papel-estrategico-do-epm-processor-na-transformacao-digital/>. Acesso em: 20 maio 2026.
- MODBUS ORGANIZATION. **Modbus Messaging on TCP/IP Implementation Guide**. Versão 1.0b. Hopkinton, MA, 2012. Disponível em: https://www.modbus.org/docs/Modbus_Messaging_Implementation_Guide_V1_0b.pdf. Acesso em: 15 maio 2026.
- MODBUS ORGANIZATION. **Modbus over Serial Line Specification and Implementation Guide**. Versão 1.02. Hopkinton, MA, 2012. Disponível em: https://www.modbus.org/docs/Modbus_over_serial_line_V1_02.pdf. Acesso em: 15 maio 2026.

MONOSTORI, L. *et al.* Cyber-physical systems in manufacturing. **CIRP Annals**, v. 65, n. 2, p. 621–641, 2016. DOI: 10.1016/j.cirp.2016.06.005.

PEDROSA, Paulo Henrique C.; NOGUEIRA, Tiago. Computação em Nuvem. Trabalho acadêmico (disciplina G04). [S. l.], 2011.

PISCHING, Marcos André. **Arquitetura para descoberta de equipamentos em processos de manufatura com foco na indústria 4.0**. 2017. Tese (Doutorado em Engenharia de Controle e Automação Mecânica) – Escola Politécnica, Universidade de São Paulo, São Paulo.

SALVADOR, Marcelo Barbosa. **Aquisição de Dados Críticos e Redundância em Sistemas SCADA**. Elipse Software. 25 mar. 2019. Disponível em: <https://kb.elipse.com.br/aquisicao-de-dados-criticos-e-redundancia-em-sistemas-scada/>. Acesso em: 20 maio 2026.

SALVADOR, Marcelo Barbosa. **Guia de Virtualização Elipse Software**. Atualizado em 4 set. 2024. Elipse Software. 25 mar. 2019. Disponível em: <https://kb.elipse.com.br/guia-de-virtualizacao-elipse-software/>. Acesso em: 20 maio 2026.

SILVA, Marco Antonio Gomes. **Boas práticas no desenvolvimento e gestão das aplicações baseadas em Elipse E3**. Atualizado em 26 mar. 2024. Elipse Software. 22 mar. 2023. Disponível em: <https://kb.elipse.com.br/boas-praticas-no-desenvolvimento-e-gestao-das-aplicacoes-baseadas-em-elipse-e3/>. Acesso em: 20 maio 2026.

STANKOVIC, John A. Research Directions for the Internet of Things. **IEEE Internet of Things Journal**, v. 1, n. 1, p. 3–9, 2014. DOI: 10.1109/JIOT.2014.2312291.

ZANGHI, Eric. **Sistemas SCADA: Conceitos**. Porto, Portugal, 2019. Série Proteção e Comunicação de Sistemas Elétricos de Potência (PROTCOM).